
Security Review Report

NM-0901 - ArcX



NETHERMIND
SECURITY

(June 04, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	4
4	System Overview	5
4.1	Architecture	5
4.2	Registry and Deployment Control Plane	5
4.3	VaultToken (VT)	5
4.4	Strategy Layer: ST/EPT Decomposition	5
4.5	Orderbook and Matching Engine	6
4.6	Points Across VT and EPT	6
4.7	Cross-Chain Liquidity and Wildcat Integration	6
4.8	Roles, Permissions, and Upgrades	6
4.9	Operational Lifecycle (End-to-End)	6
4.10	Design Strengths and Tradeoffs	6
5	Risk Rating Methodology	8
6	Issues	9
6.1	[High] Component-level burn-from-lock bypasses Core-only burn restriction	9
6.2	[High] Increasing the performance fee results in dilution of user reward points	10
6.3	[Medium] Artificial shares registration extends the queue	10
6.4	[Low] Incorrect fee accounting	11
6.5	[Low] Missing role rotation mechanisms for the depositor and operator addresses in the OperatorVault contract	12
6.6	[Low] Missing validation for zero-size orders allows permissionless order book pollution	13
6.7	[Low] Retroactive deposit lock updates can unexpectedly trap user funds	14
6.8	[Low] Stale amount during withdrawal request creation of fees	14
6.9	[Low] The claim_all_points(...) function does not check the return value of transfer(...)	15
6.10	[Low] The constructor(...) function in the OperatorVault contract does not enforce role separation between the depositor, operator, and owner	16
6.11	[Low] The handleReceiveFinalizedMessage(...) function lacks reentrancy protection	17
6.12	[Low] The mint_st_ept(...) function allows for new token decomposition after strategy maturity	18
6.13	[Low] The redeem_matured_st(...) function bricks strategy state by breaking the st_supply == ept_supply invariant	19
6.14	[Low] Unsafe minting of withdrawal NFTs to non-compatible contracts prevents token burning and permanently locks funds	20
6.15	[Info] Asymmetric token pause configuration for ST and EPT tokens	21
6.16	[Info] Modifying the claim fee basis points bricks the contract accounting logic	22
6.17	[Info] Natspecs does not align with behaviour	23
6.18	[Info] Redundant storage write in add_vt(...) function	23
6.19	[Info] The max_slippage parameter is unbounded and can disable slippage protection	24
6.20	[Info] The update_limit_apr(...) function does not validate the status of the order	24
6.21	[Info] Typo in vtoken_stragies storage variable name	24
6.22	[Best Practices] The VaultToken contract defines an unused admin storage variable	25
6.23	[Best Practices] The receive_withdraw_funds(...) function lacks access control allowing permissionless settlement of in-transit ledger	26
6.24	[Best Practices] Unused PointTokenRedeemed event struct in OperatorVault	27
6.25	[Best Practices] Unused _sent_amount variable in the BridgeProxy contract	27
7	Documentation Evaluation	28
8	Test Suite Evaluation	29
8.1	Tests Output	29
9	About Nethermind	47

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for [Karnot's Arcx](#), a DeFi protocol that allows users to split strategies into separate yield and rewards exposure, then trade each side independently.

Arcx is a modular protocol on Starknet that combines a vault and points system, which allows yield vaults to split their yield and rewards component into two tokens and then trade them on an internal orderbook. It functions as a yield decomposition primitive that splits a yield-bearing asset into two distinct, tradable components based on the fundamental invariant represented as $1 \text{ vToken} = 1 \text{ ST} + 1 \text{ EPT}$.

When a user deposits capital for yield generation, the resulting **vault token** (vToken) can be decomposed into a **Strategy Token** (ST), which represents the principal-plus-yield leg redeemable at maturity, and an **Exchange Points Token** (EPT), which captures all accrued reward points. This architecture allows users to unbundle their investment risks to execute targeted financial strategies, such as selling EPT to lock in an upfront fixed interest rate, purchasing ST at a market discount to secure cheaper principal, or trading either piece independently based on speculation.

The protocol executes these strategies through an automated cross-chain pipeline that bridges deposited USDC from Starknet to the Ethereum EVM via Circle's **Cross-Chain Transfer Protocol** (CCTP) using atomic hook-data. Upon reaching the EVM, this capital is delegated directly to **Wildcat** lending markets to generate the underlying strategy yield, while the trading of the ST and EPT pieces occurs back on Starknet through a permissionless, APR-quoted orderbook.

This orderbook supports standard same-asset swaps as well as a cross-asset matching engine that can simultaneously pair independent ST and EPT buyers to mint and distribute fresh token pairs on the fly, before eventually routing withdrawal requests back through the CCTP bridge at maturity.

Net Asset Value (NAV) accrues continuously via a credit-index model, with protocol performance fees taken from both interest and finalized points to fund a configurable treasury.

The entire system is governed through clearly segregated roles, dividing upgrade and fee authority to the **Owner**, strategy execution to the **Operator**, capital funding to the **OperatorVault**, and emergency stop capabilities to designated **Pausors**.

The audit comprises 5230 lines of cairo code and 169 lines of solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report twenty five points of attention, where two are classified as High, one is classified as Medium, eleven are classified as Low and eleven are classified as Informational or Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

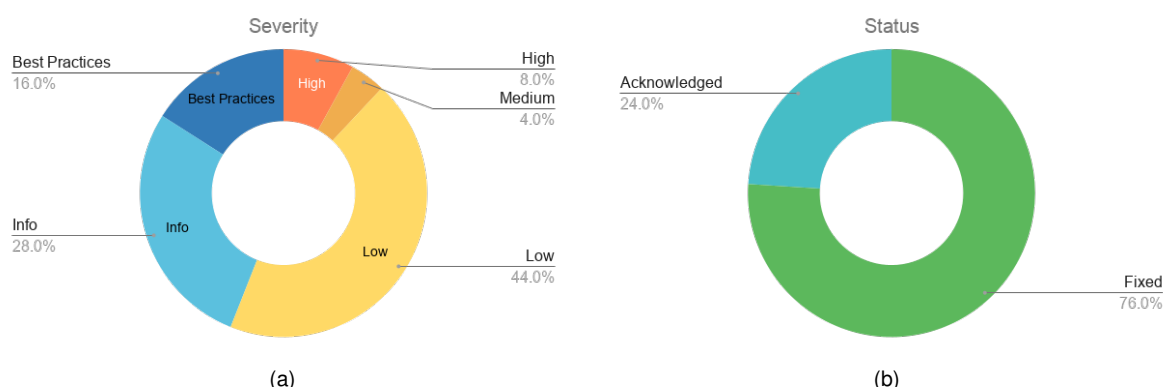


Fig. 1: Distribution of issues: Critical (0), High (2), Medium (1), Low (11), Undetermined (0), Informational (7), Best Practices (4). Distribution of status: Fixed (19), Acknowledged (6), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	June 01, 2026
Final Report	June 04, 2026
Initial Commit	01b6d8c2c799aa45a96701f27398523166061e21
Final Commit	8b97dda00f072cac05047a3855afcc91f373d764
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	lib.cairo	69	11	15.9%	1	81
2	constants.cairo	12	0	0.0%	1	13
3	point_token/interface.cairo	14	1	7.1%	1	16
4	point_token/contract.cairo	192	37	19.3%	33	262
5	ept/interface.cairo	16	1	6.2%	1	18
6	ept/contract.cairo	321	65	20.2%	47	433
7	identity/interface.cairo	3	1	33.3%	0	4
8	bridge/interface.cairo	34	20	58.8%	10	64
9	bridge/cctp_interfaces.cairo	27	2	7.4%	3	32
10	bridge/utls.cairo	59	7	11.9%	7	73
11	bridge/contract.cairo	289	75	26.0%	60	424
12	bridge/types.cairo	35	2	5.7%	5	42
13	components/user_credit/component.cairo	353	42	11.9%	39	434
14	components/user_credit/interface.cairo	30	12	40.0%	12	54
15	components/user_credit/types.cairo	23	15	65.2%	3	41
16	components/locking/component.cairo	263	69	26.2%	38	370
17	components/locking/interface.cairo	21	1	4.8%	2	24
18	vault/interface.cairo	39	3	7.7%	6	48
19	vault/contract.cairo	400	83	20.8%	74	557
20	registry/interface.cairo	41	2	4.9%	2	45
21	registry/contract.cairo	390	51	13.1%	50	491
22	registry/types.cairo	28	2	7.1%	2	32
23	vt/withdrawal_nft_interface.cairo	8	8	100.0%	5	21
24	vt/interface.cairo	72	12	16.7%	6	90
25	vt/withdrawal_nft.cairo	52	21	40.4%	12	85
26	vt/contract.cairo	1198	163	13.6%	192	1553
27	vt/types.cairo	21	3	14.3%	2	26
28	core/interface.cairo	54	14	25.9%	18	86
29	core/contract.cairo	940	201	21.4%	139	1280
30	core/types.cairo	92	28	30.4%	15	135
31	strategy/interface.cairo	7	5	71.4%	1	13
32	strategy/contract.cairo	127	30	23.6%	16	173
	Total	5230	987	18.9%	803	7020

	Contract	LoC	Comments	Ratio	Blank	Total
1	VaultReceiver.sol	169	54	32.0%	38	261
	Total	169	54	32.0%	38	261

3 Summary of Issues

	Finding	Severity	Update
1	Component-level burn-from-lock bypasses Core-only burn restriction	High	Fixed
2	Increasing the performance fee results in dilution of user reward points	High	Fixed
3	Artificial shares registration extends the queue	Medium	Acknowledged
4	Incorrect fee accounting	Low	Fixed
5	Missing role rotation mechanisms for the depositor and operator addresses in the OperatorVault contract	Low	Fixed
6	Missing validation for zero-size orders allows permissionless order book pollution	Low	Fixed
7	Retroactive deposit lock updates can unexpectedly trap user funds	Low	Acknowledged
8	Stale amount during withdrawal request creation of fees	Low	Fixed
9	The claim_all_points(...) function does not check the return value of transfer(...)	Low	Fixed
10	The constructor(...) function in the OperatorVault contract does not enforce role separation between the depositor, operator, and owner	Low	Fixed
11	The handleReceiveFinalizedMessage(...) function lacks reentrancy protection	Low	Fixed
12	The mint_st_ept(...) function allows for new token decomposition after strategy maturity	Low	Fixed
13	The redeem_matured_st(...) function bricks strategy state by breaking the st_supply == ept_supply invariant	Low	Fixed
14	Unsafe minting of withdrawal NFTs to non-compatible contracts prevents token burning and permanently locks funds	Low	Acknowledged
15	Asymmetric token pause configuration for ST and EPT tokens	Info	Acknowledged
16	Modifying the claim fee basis points bricks the contract accounting logic	Info	Fixed
17	Natspecs does not align with behaviour	Info	Fixed
18	Redundant storage write in add_vt(...) function	Info	Fixed
19	The max_slippage parameter is unbounded and can disable slippage protection	Info	Fixed
20	The update_limit_apr(...) function does not validate the status of the order	Info	Fixed
21	Typo in vtoken_stragies storage variable name	Info	Fixed
22	The VaultToken contract defines an unused admin storage variable	Best Practices	Fixed
23	The receive_withdraw_funds(...) function lacks access control allowing permissionless settlement of in-transit ledger	Best Practices	Acknowledged
24	Unused PointTokenRedeemed event struct in OperatorVault	Best Practices	Fixed
25	Unused _sent_amount variable in the BridgeProxy contract	Best Practices	Acknowledged

4 System Overview

Arcx is a modular protocol on Starknet that combines a yield-bearing USDC vault with a strategy-based orderbook for trading the vault's principal and yield separately. The protocol is designed with two surfaces that can work together without being tightly coupled. **ArcxCore** runs the orderbook and uses VaultToken (VT) balances as the collateral and settlement layer for strategy trades. The supporting contracts each handle one concern. **ArcxRegistry** is the canonical directory of exchanges, vaults, and exchange point tokens, and routes each vault to its EVM market. **VaultToken** is the ERC-4626 vault that holds user deposits and tracks NAV. **ExchangePointToken** is the registry-controlled, redemption-enabled token representing exchange points. **WithdrawalNFT** is the receipt for queued withdrawals. Bridge adapters move USDC between Starknet and EVM-based Wildcat lending markets. Each of these components operates independently and can be upgraded without disrupting the rest of the system, with child tokens such as ExchangePointToken and WithdrawalNFT upgraded through their controlling parent rather than directly.

At a high level, Arcx gives users three capabilities:

- hold and manage vault exposure through VT,
- split/recombine that exposure into strategy legs (ST/EPT),
- trade those legs in an APR-based orderbook with onchain settlement.

4.1 Architecture

Vault + Points Surface: The Vault and Points Surface act as the protocol's liquidity layer, managing user deposits and rewards. It handles standard ERC-4626 vault accounting, registers exchanges, and tracks cross-chain asset movements. Finally, it uses specialized. Receipt NFTs to securely manage users' asynchronous withdrawal processes.

Strategy + Orderbook Surface: The Strategy and Orderbook Surface sits above the vaults to run the core trading engine and yield strategies. It hosts an APR-based order book that matches trades and breaks down assets so that principal and yield trade separately. This surface handles the activation of new strategies and manages the splitting or merging of tokens.

Separation Model: The Separation Model maintains a strict boundary between the asset vaults and the trading system. Vault contracts require zero knowledge of active trading strategies, meaning the deposit system continues to work even if trading stops. All complex token splitting and merging is handled externally by the trading engine to keep vaults secure

4.2 Registry and Deployment Control Plane

The ArcxRegistry is the central source of truth for the vault system, keeping all protocol data accurate and consistent. It handles the registration of exchanges, deploys point tokens, and creates Vault Tokens (VTs) along with their withdrawal NFTs. By mapping each vault to its exact market path, it serves as the ultimate reference point for the entire network.

Strict rules ensure each exchange has only one point token and each market has only one Vault Token to prevent confusion. Cross-chain bridges pull routing data directly from this registry instead of trusting user-supplied addresses, which eliminates major security risks and errors. Finally, it includes governance controls to pause the system and protect assets during emergencies.

4.3 VaultToken (VT)

VaultToken (VT): is an ERC-4626 token that handles standard vault deposits while managing two extra features: reward points system and a secure, multi-step withdrawal process.

Deposits and NAV (Net Asset Value): When you deposit funds, the system instantly recalculates the vault's exact value so your shares are accurately priced. The vault's value grows at a fixed APR applied at specific intervals, allowing your earnings to compound over time.

Credit Periods and Points: The token automatically tracks user rewards over set time blocks. Once a period ends and the calculations are finalized, you can claim exchange point tokens proportional to your accumulated credits, ensuring payouts are always accurate and unchangeable.

Withdrawals: Exiting the vault is a multi-step process rather than an instant transaction. When you request a withdrawal, your shares are locked, and you receive a WithdrawalNFT as an on-chain receipt. Once the mandatory waiting period ends, you trade this NFT for your funds. Additionally, new deposits must clear a minimum lockup period before they can enter the withdrawal pipeline.

4.4 Strategy Layer: ST/EPT Decomposition

For each strategy, **ArcxCore** creates and manages two tokens: ST and EPT. ST is the principal-and-yield side: it redeems 1:1 for the underlying VT at maturity, so it carries both the principal and the vault's NAV yield. EPT is the points/rewards side: it accrues the credits that convert into exchange points, and it never claims the underlying principal. Core can split VT exposure into these two tokens (`mint_st_ept`) and merge them back into VT (`burn_st_ept`). This gives users flexibility: instead of holding VT alone, they can trade ST or EPT separately based on their market view.

Core uses a simple structure for this. Every strategy gets a unique, increasing `strategy_id`. The ST and EPT token contracts for that strategy are deployed under the same id, so lookup and tracking are straightforward. A strategy's maturity (`end_time`) must align with the VT period schedule, which keeps strategy settlement aligned with vault period boundaries.

4.5 Orderbook and Matching Engine

In Arcx, every order belongs to one specific strategy. An order always includes which token is traded (ST or EPT), whether it is a BUY or SELL, the APR, the size, and the `strategy_id`. This keeps each order clear and tied to a single strategy context.

Arcx does not fund orders with a normal `transferFrom`. Instead, it uses a lock model (`approve_to_lock + lock`). The token that is locked depends on the order: a BUY locks VT, a SELL ST locks ST, and a SELL EPT locks EPT. If an order is only partly filled, the remaining amount stays locked until it gets matched later or the user cancels the order.

Matching supports four cases: ST buy/sell, EPT buy/sell, cross-asset buy+buy, and cross-asset sell+sell. Same-asset matches trade tokens directly. Cross-asset BUY+BUY follows the mint path (creates ST/EPT from VT), while cross-asset SELL+SELL follows the burn path (recombines ST/EPT back to VT). Settlement depends on APR and remaining time, and always uses the maker's rate.

The `matching_time` is provided by the caller, but it is strictly checked. It must be close to the current block time (within roughly two minutes of skew), and the strategy must still be active and not matured at that time. So the protocol allows small timing flexibility, but blocks arbitrary timestamps.

4.6 Points Across VT and EPT

Points are created at the exchange point token level. But users cannot claim them immediately. Points become claimable only after VT Closes and finalizes the related credit period. This keeps point accounting tied to one period-based source of truth. For VT holders, the claim flow is simple. They claim directly through VT. VT already tracks credit index, periods, and each holder's share. So it can calculate and pay claims on its own. For EPT holders, claims go through the EPT contract. EPT does not generate points on its own. Core funds EPT claims using the correct share of points that originated from the VT settlement, so EPT holders receive equivalent economic value. This creates clear role separation. VT is the system's credit and point finalization engine. EPT is only a strategy-level distribution layer, and it depends on Core funding. So points remain unified at the VT level while still being distributed via strategy tokens.

4.7 Cross-Chain Liquidity and Wildcat Integration

The bridge flow uses four main parts. BridgeProxy handles cross-chain actions on Starknet, and VaultReceiver handles them on EVM. CCTP carries both funds and messages between chains. Wildcat is the EVM market where bridged liquidity is deployed or withdrawn.

`rebalance_liquidity` decides which way funds should move. If Starknet has extra deposits, the excess is bridged to EVM so it can earn yield. If Starknet has more mature withdrawals than local liquidity, funds are requested back from EVM to pay users.

The protocol uses a netting-first approach. It first offsets local deposits and local withdrawal needs, then bridges only the leftover difference. This reduces unnecessary bridge transfers, lowers messaging cost, and limits delay risk. Routing is also constrained by registry-approved mappings, so funds move only through approved VT-to-market paths.

4.8 Roles, Permissions, and Upgrades

- owner: configuration + upgrade authority,
- operator: runtime operations (e.g., VT liquidity rebalancing), child token upgrades are intentionally mediated through parent contracts (registry/core/VT) rather than direct EOA admin on child tokens.

This creates an explicit authority graph and reduces ambiguous privilege surfaces.

4.9 Operational Lifecycle (End-to-End)

The owner registers the exchange and deploys the VT stack via the registry. The core owner creates a strategy for a VT, ST/EPT are deployed, and the strategy is activated. Users deposit base assets into VT. Users place strategy orders after lock approvals. Matcher executes compatible order pairs, settlement moves/locks/mints/burns accordingly. VT periods finalize points, VT and EPT holders claim through their respective flows. VT withdrawal queue processes exit into NFTs, then redeems as liquidity becomes available. Operator/owner rebalances liquidity and bridges residual flows cross-chain.

4.10 Design Strengths and Tradeoffs

Strengths

- clear modular boundary between vault and strategy systems,
- deterministic and registry-driven deployment/routing control plane,
- flexible APR orderbook with native decomposition/recomposition,
- asynchronous withdrawal model aligned with bridged liquidity realities,
- explicit role and upgrade governance.

Tradeoffs

- complexity of multi-contract synchronization (credits, points, matching, bridge state),

- reliance on offchain matcher behavior for order selection/execution,
- asynchronous withdrawal UX versus immediate exit simplicity,
- pooled collateral semantics in Core require careful invariant monitoring.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [High] Component-level burn-from-lock bypasses Core-only burn restriction

File(s): [src/components/LockingComponent.cairo](#)

Description: The ST and EPT token contracts gate their burn entrypoint with an assertion ensuring that only the ArcxCore contract can call them. This mechanism ensures that the supply of both tokens moves only in lockstep through the ArcxCore decomposition and recomposition flows. These token contracts embed the LockingComponent to handle token locking capabilities.

However, the LockingComponent exposes the `burn_from_using_lock(...)` function in its public ABI. This function bypasses the contract-level core restriction by calling the internal `burn(...)` function of the ERC20 component directly underneath the contract-level access control check. Because the public locking primitives allow any user to authorize themselves as a locker by calling `approve_to_lock(...)` and `lock(...)` on their own balance, any standard token holder can establish a self-lock and invoke the public `burn_from_using_lock(...)` function.

```
1 // ...
2 // @audit-issue The public burn_from_using_lock function bypasses host access controls.
3 fn burn_from_using_lock(ref self: ContractState, ...) {
4     // ...
5     self._consume_locked_amount(...);
6     self.erc20_component.burn(...);
7 }
```

This means that an attacker can destroy one leg of the pair by burning ST tokens without burning EPT tokens, or vice versa entirely outside of the `_burn_st_ept(...)` flow from the ArcxCore contract. This desynchronizes the two token supplies. Because the `assert_strategy_collateralization(...)` function enforces that `st_supply` equals `ept_supply` at the end of the `_mint_st_ept(...)` and `_burn_st_ept(...)` flows, any future symmetric minting, burning, or cross-asset BUY and BUY match settlements will permanently revert. An attacker can permanently disable the core functionalities of an affected strategy at the cost of a single dust unit of 1, leading to a complete denial-of-service (DoS) against the strategy's operational liveness.

Recommendation(s): Consider removing the `burn_from_using_lock(...)` function from the public embeddable ABI of the LockingComponent so that it cannot be accessed externally.

Status: Fixed

Update from the client: 1. On core, we called `transfer_from_using_lock` wherever `burn_from_using_lock` was being called 2. Then we called `burn` on the ST/EPT directly 3. Burn in ST/EPT ensures caller can only be core + owner of tokens must also be core

Commit: [8b97dd](#)

6.2 [High] Increasing the performance fee results in dilution of user reward points

File(s): `src/vt/contract.cairo`

Description: The VaultToken contract coordinates yield strategies by partitioning operational timelines into distinct, sequential execution periods. Throughout any active period, user allocations generate rewards which are accumulated as credits. Once a period concludes, the contract owner finalizes the distribution by invoking the `submit_period_points(...)` function to record the gross exchange points earned, deduct a performance fee for the treasury, and distribute the remaining net points to users.

The owner has the ability to configure the performance fee any time by calling `set_performance_fees(...)` function. When the owner calls `set_performance_fees(...)`, it first settles NAV under old NAV-fee which only protects the NAV-side fee.

```

1 fn set_performance_fees(ref self: ContractState, new_performance_fees: PerformanceFees) {
2     // ...
3     self._update_nav();
4     // @audit The new configuration rate is written directly to the global storage state.
5     self.performance_fees.write(new_performance_fees);
6 }

```

The points side fee has no analogous to NAV because points aren't continuous, they're submitted in lumps via `submit_period_points(...)`. There's no equivalent of `_update_nav()` for the points side. So, when the owner subsequently processes an older completed period using the `submit_period_points(...)` function, the current logic applies this newly updated performance fee rate retroactively.

```

1 fn submit_period_points(ref self: ContractState, period_id: u64, final_points: u256) {
2     // ...
3     let total_credits = p.period_total_credits;
4     if final_points > 0 {
5         assert(total_credits > 0, 'no credits for points');
6     }
7     // @audit-issue The contract reads the live rate instead of the rate from the period.
8     let performance_fees = self.performance_fees.read();
9     let fee_points = bps_mul_down(final_points, performance_fees.points);
10    let total_allocated_points_pre_fees = final_points;
11    let net_points = final_points - fee_points;
12    // ...
13 }

```

This introduces a gap where increasing the performance fee directly results in an unintended financial benefit for the owner. Because the new rate applies when `submit_period_points(...)` is called, assuming the performance fee is increased, this forces the historical points batch to yield a substantially larger fee for the treasury, which were for the users.

As a result, increasing the performance fee results in dilution of user rewards.

Recommendation(s): Consider the implementation to make the new fee effective from future period and not immediately.

Status: Fixed

Update from the client: Fixed by removing the mutable `set_performance_fees` admin setter. VT performance fees are now deployment-time configuration. Commit: [bfc64085](#)

6.3 [Medium] Artificial shares registration extends the queue

File(s): `src/vt/contract.cairo`

Description: During the fee withdrawal request registration in `request_withdrawal()` the artificial shares are added to the `total_withdraw_requested_shares_global`, effectively increasing the `cumulative_end_shares` in the watermark check during request processing:

```

1 if request.cumulative_end_shares > self._get_processable_shares_watermark() {
2     return Option::None;
3 }

```

The purpose of the check is to protect against the immediate withdrawals after deposit. User's shares increase the left side during request creation, and the right side after deposit lock period pass or after withdrawal request is fulfilled. However, the artificial shares do not increase the right side in case of deposit lock passing, because they are never created. At the time of the check the artificial shares only increase left side, extending the deposit lock period.

Recommendation(s): Consider excluding the artificial fee shares from the deposit lock period mechanism.

Status: Acknowledged

Update from the client: Acknowledged. Fee withdrawal requests are owner-created, and the owner will only request when the withdrawal queue has sufficient capacity.

6.4 [Low] Incorrect fee accounting

File(s): [src/vt/contract.cairo](#)

Description: The owner fees are tracked differently depending on where they are stored. During every `_update_nav()` they are stored in `accrued_performance_fees_claimable` as USD value:

```
1 if fee_rate > 0 && total_share_supply > 0 {
2     let fee_delta = fee_rate
3     * (now - last_nav_update_time).into()
4     * total_share_supply
5     / SCALE;
6     self
7     .accrued_performance_fees_claimable
8     .write(self.accrued_performance_fees_claimable.read() + fee_delta);
```

However, when the owner creates the fee withdrawal request, the fees are stored as the shares during the queue:

```
1 if user == self.ownable.owner() {
2     let requested_assets = preview_assets(@self, shares);
3     let total_fees = self.accrued_performance_fees_claimable.read();
4     assert(total_fees >= requested_assets, 'Not enough fees');
5     self.accrued_performance_fees_claimable.write(total_fees - requested_assets);
```

On withdrawal processing the shares are converted to value with the increase captured during this waiting period. This leads to incorrect behaviour, since the increase of stored fees is dependent on where the fees are stored. The whole stored value of shares and fees should together represent the value deposited to Wildcat, where the funds are not split between shares value and fees.

Recommendation(s): Consider unifying the fees growth across the protocol to match the growth of funds deposited to Wildcat.

Status: Fixed

Update from the client: Fixed by tracking NAV performance fee entitlement in share units and debiting owner fee withdrawal requests from that share bucket. Commit: [bfc64085](#)

6.5 [Low] Missing role rotation mechanisms for the depositor and operator addresses in the OperatorVault contract

File(s): [src/vault/contract.cairo](#)

Description: The OperatorVault contract establishes an access control hierarchy mapping operational privileges to specific addresses. The depositor address handles underlying capital movements, such as sourcing USDC transfers via standard token allowances, while the operator address manages specialized active execution strategies. Administrative control over structural parameters is maintained through the OpenZeppelin two-step ownership component.

However, the contract makes the depositor and operator addresses entirely immutable after deployment. The values are initialized during contract construction and can only be accessed via public getter functions.

```
1 // ...
2 #[constructor]
3 fn constructor(
4     ref self: ContractState,
5     depositor: ContractAddress,
6     operator: ContractAddress,
7     owner: ContractAddress,
8     usdc: ContractAddress,
9     core_contract: ContractAddress,
10 ) {
11     // ...
12     self.ownable.initializer(owner);
13     // @audit-issue The roles are set once here and have no corresponding setter functions.
14     self.depositor.write(depositor);
15     self.operator.write(operator);
16     // ...
17 }
```

Because OperatorVault does not provide any setter function for its administrative roles, the system is exposed to an operational risk. If a private key associated with an operator or depositor is compromised, the project owner has no direct, on-chain method to revoke or rotate the vulnerable key.

Recommendation(s): Consider adding owner-restricted `set_depositor(...)` and `set_operator(...)` functions to allow rotation of address when needed.

Status: Fixed

Update from the client: Fixed by adding owner-only depositor and operator rotation mechanisms. Commit: [bfc64085](#)

6.6 [Low] Missing validation for zero-size orders allows permissionless order book pollution

File(s): `src/core/contract.cairo`

Description: The `_create_order(...)` function from the Core contract allows users to create and register new orders within the protocol. This functionality is designed to validate several parameters before persisting an order, such as checking that the caller address is non-zero, ensuring the chosen strategy is active and unmatured, and verifying that the specified apr is greater than 0. When a valid buy order is processed, the contract triggers an asset locking mechanism to secure the required tokens from the caller.

However, the `_create_order(...)` function does not validate whether the `order.size` parameter is greater than 0. This omission allows an caller to permissionlessly create an order with an `order.size` of 0.

```

1  fn _create_order(ref self: ContractState, order: OrderParams) -> Order {
2      let caller = starknet::get_caller_address();
3      assert!(caller.is_non_zero(), "Caller address is zero");
4      assert!(order.apr > 0, "APR must be greater than zero");
5      // @audit-issue The order.size parameter is never validated to be greater than 0.
6      self.assert_apr_within_max_order_apr_bps(order.apr);
7      let strategy = self
8          .assert_active_unmatured_strategy(
9              order.strategy_id, starknet::get_block_timestamp(),
10             );
11     let order_id = self.next_order_id.read();
12     self.next_order_id.write(order_id + 1);
13     // ...
14     if order.side == Side::BUY {
15         // @audit A size of 0 passes through the lock function without requiring allowance.
16         ILockingDispatcher { contract_address: strategy.vtoken }
17             .lock(caller, order.size);
18     } else {
19         // ...
20     }
21 }
```

Once created, these zero-size orders cannot be removed from the system because the `cancel_order(...)` function requires the remaining order size to be strictly greater than 0:

```

1  fn cancel_order(...) {
2      // ...
3      let matched = order.matched_amount.unwrap_or_default();
4      // @audit-issue When size is 0, remaining equals 0, causing the assertion to revert.
5      let remaining = order.size - matched;
6      assert!(remaining > 0, "Order has no remaining amount");
7  }
```

Recommendation(s): Consider adding an explicit validation check at the beginning of the `_create_order(...)` function to ensure that `order.size` is strictly greater than 0.

Status: Fixed

Update from the client: Fixed by rejecting zero-size orders before they can be inserted into the order book. Commit: [bfc64085](#)

6.7 [Low] Retroactive deposit lock updates can unexpectedly trap user funds

File(s): [src/vt/contract.cairo](#)

Description: The `_get_withdrawable_supply(...)` function from the `VaultToken` contract restricts how quickly fresh deposits can be withdrawn by calculating a global lock window based on recent daily inflows. This mechanism ensures users cannot deposit and immediately withdraw funds.

However, the `set_deposit_lock_days(...)` function from the `VaultToken` contract allows the owner to change the `deposit_lock_days` parameter dynamically without performing any validation against the existing historical state. Because the `_get_withdrawable_supply(...)` function relies on the `deposit_lock_days` at the moment of execution, any modification applies retroactively to all historic daily inflows still recorded in the system.

In contrast, the `set_withdraw_batch_delay(...)` function from the `VaultToken` contract, implements an explicit guards to prevent modifications while pending withdrawals exist.

The dynamic computation can be observed in the `_get_withdrawable_supply(...)` function:

```

1 fn _get_withdrawable_supply(self: @ContractState) -> u256 {
2     let total_supply = self.erc20.total_supply();
3     let current_day = get_block_timestamp() / SECONDS_PER_DAY;
4     // @audit Parameter is read dynamically and retroactively applied.
5     let deposit_lock_days = self.deposit_lock_days.read();
6     if deposit_lock_days == Bounded::MAX {
7         return total_supply;
8     }
9     //...
10 }
```

This means that an owner can retroactively extend the lock parameters of capital already committed under a shorter promised lock period. For example, if the initial lock duration was 1 day and is suddenly updated to 30 days, users who deposited capital under the original terms will find their deposits trapped for up to an extra 29 days if withdrawal requests approach the newly locked portion of the supply.

Recommendation(s): Consider adding a validation check within the `set_deposit_lock_days(...)` function to ensure that the parameter cannot be increased if active inflows from recent days are still present in the affected window.

Status: Acknowledged

Update from the client: Acknowledged. We generally do not intend to update deposit lock days, but we want to retain the ability to do so when required by a market maker.

6.8 [Low] Stale amount during withdrawal request creation of fees

File(s): [src/vt/contract.cairo](#)

Description: The `request_withdrawal()` function subtracts the requested fee amount from the `accrued_performance_fees_claimable` global variable:

```

1 if user == self.ownable.owner() {
2     let requested_assets = preview_assets(@self, shares);
3     let total_fees = self.accrued_performance_fees_claimable.read();
4     assert(total_fees >= requested_assets, "Not enough fees");
5     self.accrued_performance_fees_claimable.write(total_fees - requested_assets);
6 }
```

However, the value in `accrued_performance_fees_claimable` may be stale because of lack of call to `_update_nav()` during which this variable is increased by accrued fees value. During call by the owner, the revert may happen even if the amount of expected fees should be withdrawable. To get the correct amount the call to `request_withdrawal()` must be preceded by the `update_nav()`. This behaviour is different when user calls `request_withdrawal()`, since in that case there is no need to call `update_nav()` separately.

Recommendation(s): Consider refactoring the withdrawal mechanism to not being depended on calls to another public functions depending on who is calling. The possible solution would include call to `_update_nav()` in the first branch if `user == self.ownable.owner()`.

Status: Fixed

Update from the client: Fixed by checkpointing NAV before owner fee withdrawal requests read and debit `accrued_performance_fees_claimable`. Commit: [bfc64085](#)

6.9 [Low] The `claim_all_points(...)` function does not check the return value of `transfer(...)`

File(s): `src/vt/contract.cairo`

Description: The VaultToken contract allows users to claim accumulated points through the `claim_all_points(...)` function. This function identifies the point token address and attempts to transfer the allocated points to the receiver. However, the function calls `transfer(...)` on the `IERC20Dispatcher` without verifying the boolean return value.

According to the IERC20 standard, the `transfer(...)` function returns a boolean indicating whether the operation was successful. Some token implementations return false instead of reverting when a transfer fails. In such a case, the VaultToken contract would proceed as if the points were successfully distributed, leading to an inconsistent state where internal credits are cleared but the user received no tokens.

```
1 let pt = IERC20Dispatcher { contract_address: self.point_token.read() };
2 if points > 0 {
3     // @audit-issue The return value of transfer is not checked.
4     pt.transfer(receiver, points);
5 }
```

Similar issue is present in the `redeem_withdrawal(...)` function within the same contract, where the `asset.transfer(...)` call is made without asserting the result.

```
1 // @audit-issue The return value of transfer is not checked.
2 asset.transfer(caller, request.usdc_owed);
```

This lack of validation deviates from the defensive patterns used in other parts of the codebase, such as in `EPTToken`, where point transfers are explicitly asserted to ensure correctness.

Recommendation(s): Consider wrapping the `transfer(...)` calls in an `assert!(...)` statement to ensure the transaction reverts if the token contract returns false. This ensures consistency with the IERC20 standard and prevents silent failures.

Status: Fixed

Update from the client: Fixed by checking ERC20 transfer return values when claiming points. Commit: [bfc64085](#)

6.10 [Low] The constructor(...) function in the OperatorVault contract does not enforce role separation between the depositor, operator, and owner

File(s): `src/vault/contract.cairo`

Description: The OperatorVault contract is designed to facilitate split custody between different entities. According to the system design outlined in OperatorVault contract, the depositor is responsible for funding USDC while the operator handles the wrapping of assets. This operational separation is enforced throughout the contract via various access control predicates such as `assert_depositor(...)`, `assert_operator_or_depositor(...)`, and `ownable.assert_only_owner()`.

> OperatorVault: depositor funds USDC; operator wraps to strategy vToken, manages > // withdrawals/points, and submits orders on core. Access is split between depositor-only > // and operator-only entry points.

However, the `constructor(...)` only validates that the provided addresses for these roles are non-zero. It does not check if the addresses are distinct from one another.

```
1  fn constructor(  
2      ref self: ContractState,  
3      depositor: ContractAddress,  
4      operator: ContractAddress,  
5      owner: ContractAddress,  
6      // ...  
7  ) {  
8      // @audit-issue These checks ensure addresses are non-zero but do not enforce uniqueness.  
9      assert!(depositor.is_non_zero(), "Depositor is zero");  
10     assert!(operator.is_non_zero(), "Operator is zero");  
11     assert!(owner.is_non_zero(), "Owner is zero");  
12     // ...  
13  
14     self.ownable.initializer(owner);  
15     self.depositor.write(depositor);  
16     self.operator.write(operator);  
17     // ...  
18 }
```

This means that a single address can be assigned all three roles during deployment. While the contract remains functionally consistent, this bypasses the intended security model of segregated duties. If the single key holding all roles is compromised, the impact is significantly increased as the attacker would gain simultaneous control over capital provision, day-to-day operations, and administrative functions like pausing or upgrading the vault.

Recommendation(s): Consider implementing explicit checks in the `constructor(...)` to ensure that depositor, operator, and owner are unique addresses if the protocol requires segregated roles.

Status: Fixed

Update from the client: Fixed by enforcing role separation between the depositor, operator, and owner in the OperatorVault constructor. Commit: [bfc64085](#)

6.11 [Low] The `handleReceiveFinalizedMessage(...)` function lacks reentrancy protection

File(s): `solidity/src/VaultReceiver.sol`

Description: The `VaultReceiver` contract handles Cross-Chain Transfer Protocol (CCTP) messages via two entrypoints depending on message finality. The `handleReceiveUnfinalizedMessage(...)` function is utilized for unfinalized messages, while the `handleReceiveFinalizedMessage(...)` function is utilized for finalized ones. Both functions route their execution flow internally to the `_handleWithdrawRequest(...)` function to complete the withdrawal logic. However, only the unfinalized entrypoint applies the `nonReentrant` modifier. The finalized execution path lacks this guard, creating an inconsistent defense-in-depth hardening gap across two code paths that perform identical state changes.

```
1 // ...
2 function handleReceiveFinalizedMessage(
3     uint32 sourceDomain,
4     bytes32 sender,
5     uint32, /* finalityThresholdExecuted */
6     bytes calldata messageBody
7 )
8     external
9     whenNotPaused
10    returns (bool)
11 {
12     _validateMessageSender(sourceDomain, sender);
13     return _handleWithdrawRequest(messageBody);
14 }
15
16 function handleReceiveUnfinalizedMessage(
17     uint32 sourceDomain,
18     bytes32 sender,
19     uint32, /* finalityThresholdExecuted */
20     bytes calldata messageBody
21 ) external whenNotPaused nonReentrant returns (bool) {
22     _validateMessageSender(sourceDomain, sender);
23     return _handleWithdrawRequest(messageBody);
24 }
25 // ...
```

The underlying issue is that the internal `_handleWithdrawRequest(...)` function makes an external call to an external market contract to queue the withdrawal. Following this external interaction, the function updates the state of the `withdrawBatches` mapping.

Recommendation(s): Consider adding `nonReentrant` protection to `handleReceiveFinalizedMessage`

Status: Fixed

Update from the client: Fixed by adding `nonReentrant` protection to `handleReceiveFinalizedMessage`. Commit: [bfc64085](#)

6.12 [Low] The mint_st_ept(...) function allows for new token decomposition after strategy maturity

File(s): [src/core/contract.cairo](#)

Description: The Core contract provides the mint_st_ept(...) function, which allows users to decompose vToken into ST and EPT tokens. This mechanism is a core part of the protocol's lifecycle, intended to be used while a strategy is active and before it reaches its maturity date.

However, the mint_st_ept(...) function does not check if the strategy has already passed its end_time. While the order matching function correctly enforces that strategies must be active and not matured through the _create_order(...)/execute_match(...) function, a user can bypass these restrictions by interacting with the Core contract directly.

```
1  let strategy = self.assert_active_unmatured_strategy(strategy_id, matching_time_u64);
```

This creates an inconsistency where ST and EPT legs can be generated even after the strategy is matured. A user can lock vTokens to mint ST and EPT, then immediately use the redeem_matured_st(...) function to burn the newly minted ST and retrieve their vTokens.

```
1  fn mint_st_ept(
2      ref self: ContractState, strategy_id: felt252, amount: u256, recipient: ContractAddress,
3      ) {
4      self.assert_not_paused();
5      self.reentrancy_guard.start();
6      // @audit-issue This function does not check if the strategy has matured.
7  ==> let strategy = self.assert_strategy_exists(strategy_id);
8      let caller = starknet::get_caller_address();
9      ILockingDispatcher { contract_address: strategy.vtoken }.lock(caller, amount);
10     self._mint_st_ept(strategy, caller, amount, recipient);
11     self.reentrancy_guard.end();
12 }
```

After strategy is matured, a user can decompose his vToken into ST and EPT tokens 100 vToken -> 100 ST + 100 EPT.

Then call redeem_matured_st to burn ST tokens and claim back 100 vToken. The user will have 100 EPT left with him.

By performing this round-trip, the user effectively extracts a new EPT leg after the strategy's expiration leading to an unexpected increase in the EPT supply post-maturity.

Recommendation(s): Consider adding a requirement to the mint_st_ept(...) and burn_st_ept(...) functions to ensure that the current time is less than the strategy.end_time

Status: Fixed

Update from the client: Fixed by preventing mint_st_ept and burn_st_ept after strategy maturity. Commit: [bfc64085](#)

6.13 [Low] The `redeem_matured_st(...)` function bricks strategy state by breaking the `st_supply == ept_supply` invariant

File(s): `src/core/contract.cairo`

Description: The core contract enforces a strict 1:1 supply parity between ST and EPT for strategy either through composition or decomposition or Order matching. This invariant is enforced by the `assert_strategy_collateralization(...)` function, which is called at the end of most state-changing operations like minting, burning, and trade settlement.

While the trade settlement is only applicable until `end_time` of the strategy, similar enforcement is not applicable to composition or decomposition strategy.

However, once a strategy reaches maturity, users can call `redeem_matured_st(...)`. This function burns ST to return the underlying VT principal to the user but does not burn the corresponding EPT.

```
1 function redeem_matured_st(...) external {
2     // ...
3     // @audit-issue ST is burned here, reducing st_supply
4     st_token.lock(owner, amount);
5     st_token.burn_from_using_lock(owner, amount);
6
7     // @audit EPT supply remains unchanged, causing a mismatch
8     // ...
9 }
```

Because `st_supply` decreases while `ept_supply` remains constant, the first redemption of ST immediately breaks the equality check in `assert_strategy_collateralization(...)`.

```
1 function assert_strategy_collateralization(...) internal view {
2     let st_supply = ERC20ABIDispatcher { contract_address: st_token }.total_supply();
3     let ept_supply = ERC20ABIDispatcher { contract_address: ept_token }.total_supply();
4
5     // @audit-issue This assert will fail after any ST redemption
6     assert!(st_supply == ept_supply, "ST and EPT supply mismatch");
7     // ...
8 }
```

This results in a "bricked" state for the strategy. Any subsequent call to `mint_st_ept(...)`, `burn_st_ept(...)` functions that trigger the collateralization check will revert. This prevents any further decomposition or recomposition of the token legs and could potentially block other users from performing legitimate actions if those actions rely on the same validation logic.

Recommendation(s):

Status: Fixed

Update from the client: Fixed by preventing `mint_st_ept` and `burn_st_ept` after strategy maturity. Commit: [bfc64085](#)

6.14 [Low] Unsafe minting of withdrawal NFTs to non-compatible contracts prevents token burning and permanently locks funds

File(s): `src/vt/contract.cairo`, `src/vt/withdrawal_nft.cairo`

Description: The VaultToken contract utilizes an NFT receipt system to manage user withdrawals. When a user initiates a withdrawal, the protocol processes the request via the `_try_process_withdrawal(...)` function and mints a `WithdrawalNFT` to `request.user` to track their claim. Later, when the protocol is funded, the owner of the NFT must call `redeem_withdrawal(...)` to claim their USDC, which internally triggers a burn mechanism to destroy the receipt token.

```
1 // ...
2 let nft = IWithdrawalNFTDispatcher { contract_address: self.withdrawal_nft.read() };
3 // @audit-issue This internal call initiates a standard mint without checking receiver compatibility.
4 nft.mint_withdrawal(request.user, request_id);
5 // ...
```

However, the `mint_withdrawal(...)` function in the `WithdrawalNFT` contract performs a standard `mint(...)` instead of a safe minting operation. Standard minting does not verify whether the recipient address supports ERC721 tokens. If the withdrawal creator is a smart contract that does not implement the `IERC721Receiver` interface, the token will be successfully minted but completely trapped inside the receiving contract.

```
1 fn mint_withdrawal(ref self: ContractState, to: ContractAddress, token_id: u256) {
2     assert(get_caller_address() == self.vt_address.read(), "only VT");
3     // @audit-issue The contract uses standard mint instead of safe_mint.
4     self.erc721.mint(to, token_id);
5 }
```

Consequently, the protocol cannot execute the required `burn(...)` operation on the NFT, creating a deadlock where the underlying USDC remains permanently stuck in the vault.

```
1 fn redeem_withdrawal(ref self: ContractState, withdrawal_id: u256) {
2     // ...
3     // @audit The caller must be the exact owner of the NFT.
4     assert(nft.owner_of(withdrawal_id) == caller, "not NFT owner");
5     // ...
6     let nft = IWithdrawalNFTDispatcher { contract_address: self.withdrawal_nft.read() };
7     nft.burn_withdrawal(withdrawal_id);
8
9     //...
10    asset.transfer(caller, request.usdc_owed);
11    // ...
12 }
```

Recommendation(s): Considering implementing a check to ensure the depositing address supports `ERC721Receiver` interface.

Status: Acknowledged

Update from the client: It was fixed initially but we decided to revert it at the end because there was an additional risk of an attacker using non supported contracts and abusing the FIFO queue.

6.15 [Info] Asymmetric token pause configuration for ST and EPT tokens

File(s): `src/core/contract.cairo`

Description: The ArcxCore contract handles system-level pauses independently from its associated Strategy Tokens (ST) and Epoch Points Tokens (EPT). The core protocol utilizes separate administrative functions, `pause_st_ept(...)` and `unpause_st_ept(...)`, to invoke the respective `pause()` and `unpause()` entry points on the individual child token contracts.

Because the system allows these assets to be managed independently, an asymmetric pause state can occur where one token is paused while the other remains active.

In the below function, only one token address is passed.

```
1 fn pause_st_ept(ref self: ContractState, token: ContractAddress) {
2     self.ownable.assert_only_owner();
3     assert!(token.is_non_zero(), "Token is zero");
4     self.reentrancy_guard.start();
5     starknet::syscalls::call_contract_syscall(token, selector!("pause"), array![] .span())
6     .unwrap_syscall();
7     self.reentrancy_guard.end();
8 }
9
10 fn unpause_st_ept(ref self: ContractState, token: ContractAddress) {
11     self.ownable.assert_only_owner();
12     assert!(token.is_non_zero(), "Token is zero");
13     self.reentrancy_guard.start();
14     starknet::syscalls::call_contract_syscall(token, selector!("unpause"), array![] .span())
15     .unwrap_syscall();
16     self.reentrancy_guard.end();
17 }
```

If EPT is paused while ST remains unpaused, all paired operations such as `mint_st_ept(...)` and `burn_st_ept(...)` will atomically revert due to the balance updates failing on the EPT leg.

This does not introduce any direct impact.

Recommendation(s): Consider establishing an operational guideline to ensure that ST and EPT tokens are paused and unpaused atomically as a single unit per strategy identifier.

Status: Acknowledged

Update from the client: Acknowledged. We intentionally preserve the ability to pause a single token during suspicious activity while allowing unaffected token transfers to continue.

6.16 [Info] Modifying the claim fee basis points bricks the contract accounting logic

File(s): `src/core/contract.cairo`

Description: The `redeem_points(...)` function from the Core contract manages and distributes points earned by a strategy. When processing a distribution, the Core contract reads the current fee configuration from the EPTToken contract using the `get_claim_fee_basis_points(...)` function. This value is used to calculate the cumulative lifetime targets for both fees, `total_fees_should_be_charged` and net funding `total_net_should_be_funded` based on the global gross points earned.

```

1 // ...
2 let fee_bps = ept.get_claim_fee_basis_points();
3 => let total_fees_should_be_charged = total_earned * fee_bps / BASIS_POINTS;
4 => let total_net_should_be_funded = total_earned - total_fees_should_be_charged;
5 let fees_charged_so_far = self.ept_claim_fees_charged.read(ept_address);
6
7 // @audit-issue If the fee decreases, this cumulative check will fail.
8 assert!(
9     total_fees_should_be_charged >= fees_charged_so_far,
10    "Claim fee accounting regressed",
11 );
12 // ...
13 let points_funded_so_far = self.ept_points_funded.read(ept_address);
14
15 // @audit-issue If the fee increases, this cumulative check will fail.
16 assert!(
17     total_net_should_be_funded >= points_funded_so_far,
18    "EPT funding accounting regressed",
19 );
20 // ...

```

However, the EPTToken contract also implements the `set_claim_fee_basis_points(...)` function, which allows the fee configuration to be updated dynamically by the Core contract.

```

1 // ...
2 fn set_claim_fee_basis_points(ref self: ContractState, claim_fee_basis_points: u256) {
3     self.assert_not_paused();
4     assert!(get_caller_address() == self.core_address.read(), "EPT: only core");
5     assert!(claim_fee_basis_points <= BASIS_POINTS, "EPT: fee exceeds 100%");
6     // @audit-issue The fee can be modified even after points have been redeemed.
7     self.claim_fee_basis_points.write(claim_fee_basis_points);
8 }

```

If the fee configuration is modified after the `redeem_points(...)` function has already been executed at least once, it will brick the contract's accounting logic.

Depending on the direction of the fee modification, the recomputed cumulative lifetime targets will shift backward compared to the historical values already committed to storage, triggering an irreversible denial of service:

- Increasing the fee: A higher fee reduces the remaining net funding target (`total_net_should_be_funded`), causing it to drop below the historical value recorded in `points_funded_so_far`. This causes an immediate panic on the "EPT funding accounting regressed" assertion;
- Decreasing the fee: A lower fee reduces the cumulative fee target (`total_fees_should_be_charged`), causing it to drop below the historical value recorded in `fees_charged_so_far`. This causes an immediate panic on the "Claim fee accounting regressed" assertion;

Because the `before_update(...)` hook automatically triggers a call to `redeem_points(...)` during every token transfer, mint, or burn operation within the EPTToken contract, a failure in this validation step completely freezes all token liquidity. Furthermore, users will be unable to call `claim_all_points(...)` to collect rewards, leading to a lockup of funds. While the Core contract does not currently expose an external path to call `set_claim_fee_basis_points(...)`, any future upgrade adding this capability will immediately introduce this vulnerability.

Recommendation(s): Consider removing the `set_claim_fee_basis_points(...)` function from the EPTToken contract if fees are intended to remain immutable after strategy deployment. If fee adjustments are required, consider updating the accounting mechanism to handle dynamic fee modifications.

Status: Fixed

Update from the client: Fixed by removing the mutable `set_claim_fee_basis_points` setter. The EPT claim fee is now fixed at token creation. Commit: [bfc64085](#)

6.17 [Info] Natspecs does not align with behaviour

File(s): [src/vault/contract.cairo](#)

Description: While the natspec says, depositor is responsible for moving USDC funds and operator is responsible for other operations, the implementation assigns depositor and operator for operations.

```

1 fn wrap_to_vtoken_and_place_order(ref self: ContractState, order: OrderRequest) -> u64 {
2     //...
3     self.assert_operator_or_depositor();
4 }

```

> // OperatorVault: depositor funds USDC; operator wraps to strategy vToken, manages > // withdrawals/points, and submits orders on core. Access is split between depositor-only > // and operator-only entry points.

Recommendation(s): Considering correcting the Natspecs.

Status: Fixed

Update from the client: Fixed by aligning the NatSpec comments with the actual OperatorVault behavior. Commit: [bfc64085](#)

6.18 [Info] Redundant storage write in add_vt(...) function

File(s): [src/registry/contract.cairo](#)

Description: The add_vt(...) function in the ArcxRegistry contract is used by the owner to deploy and register a new VToken and its associated withdrawal NFT. During this process, the function records the newly deployed VToken details into the vtokens storage map.

However, the function contains two consecutive calls to self.vtokens.write(...) using the same vtoken address as the key and logically identical VToken struct data. The only difference between the two calls is the order in which the fields are initialized in the struct literal.

```

1 // ...
2 // @audit The first storage write occurs here.
3 self
4     .vtokens
5     .write(
6         vtoken,
7         VToken {
8             address: vtoken, exchange_id: params.exchange_id, market: params.market,
9         },
10    );
11 // @audit-issue The second storage write is redundant as it overwrites the first with the same data.
12 self
13     .vtokens
14     .write(
15         vtoken,
16         VToken {
17             exchange_id: params.exchange_id, market: params.market, address: vtoken,
18         },
19    );
20 // ...

```

Since both writes target the same storage slot with the same values, the second write is idempotent and serves no functional purpose. This redundancy increases the contract's complexity and can be confusing for developers or auditors reviewing the logic.

Recommendation(s): Consider removing one of the redundant self.vtokens.write(...) blocks to keep the code clean and maintainable.

Status: Fixed

Update from the client: Fixed by removing the redundant add_vt storage write. Commit: [bfc64085](#)

6.19 [Info] The max_slippage parameter is unbounded and can disable slippage protection

File(s): `src/core/contract.cairo`

Description: Description: The `create_market_and_match(...)` function from the Core contract allows users to create a new market and execute order matching. When submitting an order, users provide a `quoted_apr` alongside a `max_slippage` parameter denominated in basis points to establish acceptable boundaries for execution price movements.

However, the function fails to validate that the user-supplied `max_slippage` is constrained to a reasonable maximum threshold or capped at `BASIS_POINTS`. When an exceptionally large `max_slippage` value is supplied, the resulting `slippage_delta` calculation scales proportionally and can exceed the `quoted_apr` value itself. This shifts the execution boundaries to their absolute extremes, completely disabling the user's slippage safeguards.

Solidity function `createMarketAndMatch(...)` external // ... // @audit-issue The max_slippage parameter is not validated against BASIS_POINTS. `uint256 slippageDelta = (quotedApr * order.max_slippage) / BASIS_POINTS;`

if (asset == Asset.ST side == Side.BUY) if (quotedApr > slippageDelta) worstAcceptableApr = quotedApr - slippageDelta; else worstAcceptableApr = 1; else worstAcceptableApr = min(maxOrderAprBps, quotedApr + slippageDelta); // ...

If `slippageDelta` exceeds `quotedApr`, then the execution logic falls into the inner else branch and sets `worstAcceptableApr` to 1 (representing a 0.01% APR). This completely removes the price protection for the transaction, forcing the user to accept a highly disadvantageous trade where they receive significantly fewer tokens than anticipated.

Recommendation(s): Consider introducing an explicit validation check to verify that the `max_slippage` parameter does not exceed `BASIS_POINTS`.

Status: Fixed

Update from the client: Fixed by rejecting `max_slippage` values above `BASIS_POINTS`. Commit: [bfc64085](#)

6.20 [Info] The update_limit_apr(...) function does not validate the status of the order

File(s): `src/core/contract.cairo`

Description: The `update_limit_apr(...)` function is used by users to update the APR of their existing orders. However, the function does not check the current status of the order before allowing the modification.

Recommendation(s): Consider introducing a validation check ensuring that the order status is strictly set to either `OrderStatus::Open` or `OrderStatus::Partially_filled` before performing the update.

Status: Fixed

Update from the client: Fixed by allowing APR updates only for `Open` or `Partially_filled` orders. Commit: [bfc64085](#)

6.21 [Info] Typo in vtoken_stragies storage variable name

File(s): `src/core/contract.cairo`

Description: The Core contract initializes and manages strategic layouts for tracking various token mappings. However, the storage variable responsible for tracking these strategies contains a typographical error. It is defined as `vtoken_stragies` instead of `vtoken_strategies`. This misspelled storage name is subsequently utilized when accessing or updating state within the contract functions.

```
1 // ...
2 // @audit-issue The storage variable name is misspelled.
3 vtoken_stragies: Map<ContractAddress, VTokenStrategies>,
4 // ...
5
```

The state variable is later read from and written to across different parts of the functionality. For instance, when registering new tokens or executing queries, the codebase uses the misspelled identifier to resolve the underlying storage path components.

While this does not break the inner operational logic, but the variable name is consistently referenced in all active internal storage locations. Starknet storage slots are directly derived using the field identifier string. Altering this string in a future upgrade without careful manual state migrations will cause the protocol to lose references to historical storage slots.

Recommendation(s): Consider renaming `vtoken_stragies` to `vtoken_strategies` to ensure alignment with standard project naming conventions and to keep the codebase maintainable.

Status: Fixed

Update from the client: Fixed by renaming `vtoken_stragies` to `vtoken_strategies`. Commit: [bfc64085](#)

6.22 [Best Practices] The VaultToken contract defines an unused admin storage variable

File(s): [src/vt/contract.cairo](#)

Description: The VaultToken contract utilizes the OpenZeppelin OwnableComponent to manage contract ownership and restricted operations. Privileged functions within the IVaultTokenAdmin interface enforce access control by calling `self.ownable.assert_only_owner()` or verifying the authorized operator address.

However, the contract declares an admin variable within its Storage struct that is completely unused.

```
1 // ...
2 struct Storage {
3     performance_fees: PerformanceFees,
4     accrued_performance_fees_claimable: u256,
5     // @audit-issue This variable is never initialized, read, or written to in the logic.
6     admin: ContractAddress,
7     operator: ContractAddress,
8     bridge_proxy: ContractAddress,
9     treasury: ContractAddress,
10 }
11 // ...
```

The admin variable is never assigned a value in the constructor, is not exposed via any public getters or setters, and is never referenced anywhere else in the repository. Keeping dead storage slots increases confusion and risks storage alignment issues in future contract upgrades if the variable is modified or removed.

Recommendation(s): Consider removing the admin variable from the Storage struct prior to production deployment to clean up the contract layout.

Status: Fixed

Update from the client: Fixed by removing the unused VT admin storage variable. Commit: [bfc64085](#)

6.23 [Best Practices] The receive_withdraw_funds(...) function lacks access control allowing permissionless settlement of in-transit ledger

File(s): src/vt/contract.cairo

Description: The receive_withdraw_funds(...) function from the VaultToken contract is designed to pull asynchronously returned withdrawal funds into the vault after a bridge withdrawal is requested. When an operator triggers a liquidity rebalance, the intransit_funds variable is increased, and a withdrawal request is sent to the bridge proxy. Once the bridge processes the transfer, it is expected to call receive_withdraw_funds(...) to transfer the USDC tokens back to the vault, converting the in-transit ledger balance into claimable funds for users.

However, the receive_withdraw_funds(...) function does not validate that the caller is the authorized bridge proxy contract. It only verifies that the contract is not paused, satisfies the reentrancy guard, and successfully executes transfer_from(...) from the caller to the vault.

```

1 fn receive_withdraw_funds(ref self: ContractState, amount: u256) {
2     self.assert_not_paused();
3     self.reentrancy_guard.start();
4     assert(amount > 0, "zero amount");
5     let asset = IERC20Dispatcher { contract_address: self.erc4626.ERC4626_asset.read() };
6     let this = get_contract_address();
7     // @audit-issue Any caller can invoke this function as long as they provide the allowance
8     let caller = get_caller_address();
9     let balance_before = asset.balance_of(this);
10    assert(asset.transfer_from(caller, this, amount), "pull failed");
11 }

```

This lack of access control allows any external account or contract to execute permissionless settlement of the in-transit ledger. This action clears the intransit_funds state and prematurely increments total_withdraw_claimable_usdc_global. When the authentic bridge transaction is subsequently executed, the settled_amount calculation evaluates to 0 because intransit_funds has already been reduced to 0.

```

1 // ...
2 let intransit_funds = self.intransit_funds.read();
3 let settled_amount = min(amount, intransit_funds);
4 let surplus_amount = amount - settled_amount;
5 self.intransit_funds.write(intransit_funds - settled_amount);
6 self
7     .total_withdraw_claimable_usdc_global
8     .write(self.total_withdraw_claimable_usdc_global.read() + settled_amount);
9 if surplus_amount > 0 {
10    // @audit The authentic bridge return will be fully classified as surplus and sent here
11    assert(
12        asset.transfer(self.treasury.read(), surplus_amount),
13        "treasury transfer failed",
14    );
15 }
16 }

```

Consequently, the entire amount of canonical bridge return funds is treated as a surplus and redirected to the treasury address. These is no impact and funds received from the caller of receive_withdraw_funds is treated as donation. But, as this function is clearly nominated to be called by bridge proxy, it is a best practice to restrict the caller to entitled address.

Recommendation(s): Consider adding an explicit access control check to ensure that only the authorized bridge proxy address can successfully call the function.

```

1 assert(get_caller_address() == self.bridge_proxy.read(), "only bridge");

```

Status: Acknowledged

Update from the client: Acknowledged. This is intentional so the vault can be manually funded if bridge-side settlement has an issue.

6.24 [Best Practices] Unused PointTokenRedeemed event struct in OperatorVault

File(s): [src/vault/contract.cairo](#)

Description: The OperatorVault contract defines an event struct named PointTokenRedeemed to track the redemption of point tokens.

```
1 #[derive(Drop, starknet::Event)]
2 struct PointTokenRedeemed {
3     token: ContractAddress,
4     shares_in: u256,
5     assets_out: u256,
6 }
```

But, it was never used in the contract. It is considered best practice to remove unused structures and dead code to maintain a clean, readable, and maintainable codebase.

Recommendation(s): Consider removing the PointTokenRedeemed struct from the contract if it is no longer needed.

Status: Fixed

Update from the client: Fixed by removing the unused PointTokenRedeemed event struct. Commit: [bfc64085](#)

6.25 [Best Practices] Unused _sent_amount variable in the BridgeProxy contract

File(s): [src/bridge/contract.cairo](#)

Description: The receive_and_return(...) function from the BridgeProxy contract handles incoming cross-chain messages verified by Cross-Chain Transfer Protocol (CCTP). When parsing a withdrawal completion action, the function extracts transaction parameters from the message hook_data payload to complete the withdrawal and route funds.

However, the function extracts the _sent_amount variable from byte offset 65 of the payload but leaves it completely unused. The contract relies strictly on the token balance delta before and after execution to determine the authoritative amount of tokens minted.

Considering the protocol is using the standard mode, the fee applicable is 0. As a result, the _sent_amount should be equal to delta balance after execution.

Recommendation(s): Consider adding an assert for _sent_amount to be equal the delta amount.

Status: Acknowledged

Update from the client: Acknowledged. This is kept to ensure fund flow continues if we use a token w/ a different CCTP fee structure down the line. Commit: [bfc64085](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- **Technical whitepaper:** A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- **User manual:** A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- **Code documentation:** Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- **API documentation:** API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- **Testing documentation:** Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- **Audit documentation:** Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Karnot Arcx protocol documentation

The Karnot Arcx team provided documentation detailing the full protocol architecture. Additionally, the team provided a walk-through of the entire protocol including all the main flows with expected state changes. Furthermore, the team addressed all questions and concerns raised by the Nethermind Security team, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

```

Compiling test(arcxcore_unittest) arcxcore v0.1.0
↳ (/Users/admin-sandbox/audits/NM-0901-Security-Review-Karnot/Scarb.toml)
Finished `dev` profile target(s) in 51 seconds

Collected 526 test(s) from arcxcore package
Running 526 test(s) from src/
[PASS] arcxcore::tests::bridge_admin::test_deposit_reverts_when_not_initialized (l1_gas: ~0, l1_data_gas: ~192, l2_gas: ~1378140)
[PASS] arcxcore::tests::bridge_admin::test_receive_and_return_reverts_when_not_initialized (l1_gas: ~0, l1_data_gas: ~192, l2_gas: ~1439570)
[PASS] arcxcore::tests::bridge_admin::test_receive_and_return_reverts_if_registry_not_set (l1_gas: ~0, l1_data_gas: ~2496, l2_gas: ~20532140)
[PASS] arcxcore::tests::bridge_admin::test_pause_blocks_request_withdraw (l1_gas: ~0, l1_data_gas: ~7584, l2_gas: ~14436468)
[PASS] arcxcore::tests::bridge_admin::test_pause_blocks_receive_and_return (l1_gas: ~0, l1_data_gas: ~7584, l2_gas: ~14394948)
[PASS] arcxcore::tests::bridge_admin::test_bridge_get_version_returns_v0 (l1_gas: ~0, l1_data_gas: ~7488, l2_gas: ~12824948)
[PASS] arcxcore::tests::bridge_admin::test_request_withdraw_reverts_if_registry_not_set (l1_gas: ~0, l1_data_gas: ~2400, l2_gas: ~6525212)
[PASS] arcxcore::tests::bridge_admin::test_request_withdraw_reverts_when_not_initialized (l1_gas: ~0, l1_data_gas: ~192, l2_gas: ~1233370)
[PASS] arcxcore::tests::bridge_admin::test_initialize_happy_path_sets_all_fields (l1_gas: ~0, l1_data_gas: ~7488, l2_gas: ~14015218)
[PASS] arcxcore::tests::bridge_admin::test_initialize_reverts_if_not_owner (l1_gas: ~0, l1_data_gas: ~1824, l2_gas: ~4438922)
[PASS] arcxcore::tests::bridge_admin::test_initialize_reverts_on_second_call (l1_gas: ~0, l1_data_gas: ~2304, l2_gas: ~4795422)
[PASS] arcxcore::tests::bridge_admin::test_set_registry_happy_path (l1_gas: ~0, l1_data_gas: ~14976, l2_gas: ~26322916)
[PASS] arcxcore::tests::bridge_admin::test_pause_blocks_deposit (l1_gas: ~0, l1_data_gas: ~7584, l2_gas: ~14329508)
[PASS] arcxcore::tests::core_match::test_match_orders_cross_buy_buy_incompatible_aprs_reverts (l1_gas: ~0, l1_data_gas: ~13056, l2_gas: ~49752274)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_st_buy_exactly_passes_two_percent_slippage (l1_gas: ~0, l1_data_gas: ~15744, l2_gas: ~95926541)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_st_buy_reverts_when_two_percent_slippage_is_exceeded (l1_gas: ~0, l1_data_gas: ~13536, l2_gas: ~94929241)
[PASS] arcxcore::tests::bridge_admin::test_set_registry_only_owner (l1_gas: ~0, l1_data_gas: ~7488, l2_gas: ~13303258)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_reverts_when_matches_leave_remainder (l1_gas: ~0, l1_data_gas: ~13440, l2_gas: ~117848530)
[PASS] arcxcore::tests::bridge_admin::test_set_registry_rejects_zero_registry (l1_gas: ~0, l1_data_gas: ~7488, l2_gas: ~13307568)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_st_buy_fills_across_multiple_orders_within_slippage (l1_gas: ~0, l1_data_gas: ~17760, l2_gas: ~155296282)
[PASS] arcxcore::tests::bridge_admin::test_upgrade_owner_only (l1_gas: ~0, l1_data_gas: ~7488, l2_gas: ~13133368)
[PASS] arcxcore::tests::core_match::test_match_orders_cross_opposite_sides_reverts (l1_gas: ~0, l1_data_gas: ~14304, l2_gas: ~87719203)
[PASS] arcxcore::tests::core_match::test_match_orders_cross_buy_buy_mints_st_and_ept (l1_gas: ~0, l1_data_gas: ~16224, l2_gas: ~142063511)
[PASS] arcxcore::tests::bridge_deposit::test_deposit_encodes_hook_data_correctly (l1_gas: ~0, l1_data_gas: ~8640, l2_gas: ~37198888)

```

```
[PASS] arcxcore::tests::core_match::test_match_orders_cross_opposite_sides_reverts_for_sell_ept_vs_buy_st (l1_gas: ~0,
↳ l1_data_gas: ~14304, l2_gas: ~87719903)
[PASS] arcxcore::tests::bridge_deposit::test_deposit_happy_path_records_cctp_call (l1_gas: ~0, l1_data_gas: ~8832,
↳ l2_gas: ~26182057)
[PASS] arcxcore::tests::bridge_deposit::test_deposit_ids_are_monotonic (l1_gas: ~0, l1_data_gas: ~8736, l2_gas:
↳ ~47438957)
[PASS] arcxcore::tests::bridge_deposit::test_deposit_pulls_usdc_from_user (l1_gas: ~0, l1_data_gas: ~8640, l2_gas:
↳ ~25298747)
[PASS] arcxcore::tests::core_match::test_match_orders_cross_sell_sell_incompatible_aprs (l1_gas: ~0, l1_data_gas:
↳ ~14784, l2_gas: ~121741464)
[PASS] arcxcore::tests::core_match::test_match_orders_ept_incompatible_aprs_reverts (l1_gas: ~0, l1_data_gas: ~14208,
↳ l2_gas: ~87698113)
[PASS] arcxcore::tests::bridge_deposit::test_deposit_reverts_on_unregistered_vtoken (l1_gas: ~0, l1_data_gas: ~7488,
↳ l2_gas: ~13860498)
[PASS] arcxcore::tests::bridge_deposit::test_deposit_reverts_on_zero_amount (l1_gas: ~0, l1_data_gas: ~7488, l2_gas:
↳ ~13856088)
[PASS] arcxcore::tests::core_match::test_match_orders_ept_same_asset_happy_path (l1_gas: ~0, l1_data_gas: ~16800,
↳ l2_gas: ~122303148)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_mock_vtoken_allowance_is_incremental (l1_gas:
↳ ~0, l1_data_gas: ~3648, l2_gas: ~79208068)
[PASS] arcxcore::tests::core_match::test_match_orders_cross_sell_sell_apr_split (l1_gas: ~0, l1_data_gas: ~17088,
↳ l2_gas: ~183611450)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_permissionless_caller (l1_gas: ~0, l1_data_gas:
↳ ~7680, l2_gas: ~51711616)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_happy_path (l1_gas: ~0, l1_data_gas: ~12192,
↳ l2_gas: ~91637851)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_if_mt_returns_false (l1_gas: ~0,
↳ l1_data_gas: ~7488, l2_gas: ~28897336)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_on_short_hook_data (l1_gas: ~0,
↳ l1_data_gas: ~7584, l2_gas: ~26366088)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_on_market_mismatch (l1_gas: ~0,
↳ l1_data_gas: ~7584, l2_gas: ~47710996)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_on_short_message (l1_gas: ~0,
↳ l1_data_gas: ~7584, l2_gas: ~21186926)
[PASS] arcxcore::tests::core_match::test_match_orders_matching_time_skew_rejected (l1_gas: ~0, l1_data_gas: ~14208,
↳ l2_gas: ~87329611)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_cancelled_order (l1_gas: ~0, l1_data_gas: ~12960,
↳ l2_gas: ~52573146)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_different_strategies (l1_gas: ~0, l1_data_gas: ~15648,
↳ l2_gas: ~54167295)
[PASS] arcxcore::tests::core_match::test_match_orders_match_nonce_increments (l1_gas: ~0, l1_data_gas: ~18432, l2_gas:
↳ ~152550595)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_unknown_maker (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19765042)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_same_user (l1_gas: ~0, l1_data_gas: ~12480, l2_gas:
↳ ~38994756)
[PASS] arcxcore::tests::core_match::test_match_orders_partial_fill_status_transitions (l1_gas: ~0, l1_data_gas: ~16800,
↳ l2_gas: ~119460248)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_unknown_taker (l1_gas: ~0, l1_data_gas: ~11808,
↳ l2_gas: ~33107910)
[PASS] arcxcore::tests::core_redeem_points::test_redeem_points_reverts_on_zero_ept (l1_gas: ~0, l1_data_gas: ~1248,
↳ l2_gas: ~1735270)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_zero_taker_id (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19165452)
[PASS] arcxcore::tests::core_redeem_points::test_redeem_points_reverts_on_unknown_ept (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~21171546)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_on_zero_maker_id (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19163252)
[PASS] arcxcore::tests::core_match::test_match_orders_partially_filled_order_can_be_filled_again_for_remaining_amount
↳ (l1_gas: ~0, l1_data_gas: ~19008, l2_gas: ~195511760)
```

```
[PASS] arcxcore::tests::core_regressions::test_core_can_pause_and_unpause_strategy_tokens (l1_gas: ~0, l1_data_gas:
↳ ~10176, l2_gas: ~22263342)
[PASS] arcxcore::tests::core_regressions::test_core_can_pause_and_unpause_strategy_tokens_while_core_is_paused (l1_gas:
↳ ~0, l1_data_gas: ~10272, l2_gas: ~21774462)
[PASS] arcxcore::tests::core_match::test_match_orders_same_asset_same_side_reverts (l1_gas: ~0, l1_data_gas: ~13056,
↳ l2_gas: ~49775064)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_when_taker_older_than_maker (l1_gas: ~0, l1_data_gas:
↳ ~12384, l2_gas: ~35800032)
[PASS] arcxcore::tests::core_match::test_match_orders_reverts_when_strategy_is_inactive (l1_gas: ~0, l1_data_gas:
↳ ~14112, l2_gas: ~89331147)
[PASS] arcxcore::tests::core_regressions::test_core_can_upgrade_deployed_ept_token (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19936742)
[PASS] arcxcore::tests::core_regressions::test_core_can_upgrade_deployed_strategy_token (l1_gas: ~0, l1_data_gas:
↳ ~10176, l2_gas: ~19936042)
[PASS] arcxcore::tests::core_regressions::test_core_pause_blocks_add_strategy (l1_gas: ~0, l1_data_gas: ~10272, l2_gas:
↳ ~20667624)
[PASS] arcxcore::tests::core_regressions::test_core_pause_blocks_mint_st_ept (l1_gas: ~0, l1_data_gas: ~11232, l2_gas:
↳ ~32158270)
[PASS] arcxcore::tests::core_regressions::test_core_rejects_strategy_maturity_off_period_boundary (l1_gas: ~0,
↳ l1_data_gas: ~7104, l2_gas: ~14447293)
[PASS] arcxcore::tests::core_match::test_match_orders_st_incompatible_aprs_reverts (l1_gas: ~0, l1_data_gas: ~14400,
↳ l2_gas: ~87698113)
[PASS] arcxcore::tests::core_match::test_match_orders_st_same_asset_happy_path (l1_gas: ~0, l1_data_gas: ~15744,
↳ l2_gas: ~98638629)
[PASS] arcxcore::tests::core_regressions::test_paused_ept_allows_upgrade (l1_gas: ~0, l1_data_gas: ~10272, l2_gas:
↳ ~20197762)
[PASS] arcxcore::tests::core_regressions::test_ept_first_mint_after_credit_index_moves (l1_gas: ~0, l1_data_gas:
↳ ~13536, l2_gas: ~70994937)
[PASS] arcxcore::tests::core_regressions::test_ept_pause_blocks_transfer (l1_gas: ~0, l1_data_gas: ~13344, l2_gas:
↳ ~77446727)
[PASS] arcxcore::tests::core_regressions::test_ept_credits_freeze_once_strategy_matures (l1_gas: ~0, l1_data_gas:
↳ ~13536, l2_gas: ~101013605)
[PASS] arcxcore::tests::core_mint_burn::test_burn_st_ept_happy_path (l1_gas: ~0, l1_data_gas: ~11808, l2_gas: ~98673870)
[PASS] arcxcore::tests::core_mint_burn::test_burn_st_ept_reverts_on_unknown_strategy (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~20020944)
[PASS] arcxcore::tests::core_regressions::test_paused_st_allows_upgrade (l1_gas: ~0, l1_data_gas: ~10272, l2_gas:
↳ ~20196362)
[PASS] arcxcore::tests::core_mint_burn::test_burn_st_ept_reverts_on_zero_amount (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~21096324)
[PASS] arcxcore::tests::core_regressions::test_ept_ignores_vt_periods_finalized_after_maturity (l1_gas: ~0,
↳ l1_data_gas: ~16704, l2_gas: ~163665865)
[PASS] arcxcore::tests::core_regressions::test_registry_pause_blocks_core_create_order (l1_gas: ~0, l1_data_gas:
↳ ~11232, l2_gas: ~31991580)
[PASS] arcxcore::tests::core_regressions::test_public_split_uses_fresh_lock_without_consuming_order_lock (l1_gas: ~0,
↳ l1_data_gas: ~12768, l2_gas: ~71305421)
[PASS] arcxcore::tests::core_mint_burn::test_burn_st_ept_reverts_on_zero_recipient (l1_gas: ~0, l1_data_gas: ~12192,
↳ l2_gas: ~69781889)
[PASS] arcxcore::tests::core_regressions::test_registry_pause_blocks_ept_transfer (l1_gas: ~0, l1_data_gas: ~13344,
↳ l2_gas: ~77780097)
[PASS] arcxcore::tests::core_mint_burn::test_mint_st_ept_mints_equal_st_and_ept (l1_gas: ~0, l1_data_gas: ~12000,
↳ l2_gas: ~66374057)
[PASS] arcxcore::tests::core_regressions::test_registry_pause_blocks_st_transfer (l1_gas: ~0, l1_data_gas: ~13344,
↳ l2_gas: ~70237757)
[PASS] arcxcore::tests::core_mint_burn::test_mint_st_ept_pulls_vt_collateral_via_lock (l1_gas: ~0, l1_data_gas: ~12000,
↳ l2_gas: ~66626697)
[PASS] arcxcore::tests::core_mint_burn::test_mint_st_ept_reverts_on_unknown_strategy (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~20010434)
[PASS] arcxcore::tests::core_regressions::test_st_pause_blocks_transfer (l1_gas: ~0, l1_data_gas: ~13344, l2_gas:
↳ ~69903687)
```



```
[PASS] arcxcore::tests::core_mint_burn::test_mint_st_ept_asserts_collateralization_invariant_across_multiple_mints
↳ (l1_gas: ~0, l1_data_gas: ~12000, l2_gas: ~123755263)
[PASS] arcxcore::tests::core_strategy::test_activate_strategy_happy_path_flips_bit (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~20212446)
[PASS] arcxcore::tests::core_mint_burn::test_mint_st_ept_reverts_on_zero_amount (l1_gas: ~0, l1_data_gas: ~11136,
↳ l2_gas: ~31082250)
[PASS] arcxcore::tests::core_strategy::test_activate_strategy_reverts_on_unknown (l1_gas: ~0, l1_data_gas: ~7104,
↳ l2_gas: ~12320155)
[PASS] arcxcore::tests::core_strategy::test_add_strategy_deploys_st_and_ept_tokens (l1_gas: ~0, l1_data_gas: ~10080,
↳ l2_gas: ~17892008)
[PASS] arcxcore::tests::core_mint_burn::test_mint_st_ept_reverts_on_zero_recipient (l1_gas: ~0, l1_data_gas: ~11136,
↳ l2_gas: ~31119150)
[PASS] arcxcore::tests::core_strategy::test_add_strategy_happy_path_assigns_monotonic_ids (l1_gas: ~0, l1_data_gas:
↳ ~12384, l2_gas: ~19442793)
[PASS] arcxcore::tests::core_strategy::test_add_strategy_reverts_on_unknown_vtoken (l1_gas: ~0, l1_data_gas: ~7104,
↳ l2_gas: ~13195883)
[PASS] arcxcore::tests::core_strategy::test_add_strategy_reverts_when_maturity_is_not_on_period_boundary (l1_gas: ~0,
↳ l1_data_gas: ~7104, l2_gas: ~13391213)
[PASS] arcxcore::tests::core_orders::test_buy_order_locked_vtoken_collateral_blocks_transfer_above_unlocked_remainder
↳ (l1_gas: ~0, l1_data_gas: ~11904, l2_gas: ~37391330)
[PASS] arcxcore::tests::core_orders::test_cancel_order_buy_unlocks_vt (l1_gas: ~0, l1_data_gas: ~11712, l2_gas:
↳ ~35448722)
[PASS] arcxcore::tests::core_strategy::test_add_strategy_reverts_when_registry_not_set (l1_gas: ~0, l1_data_gas: ~1344,
↳ l2_gas: ~2316400)
[PASS] arcxcore::tests::core_orders::test_cancel_order_only_self (l1_gas: ~0, l1_data_gas: ~11808, l2_gas: ~33306362)
[PASS] arcxcore::tests::core_strategy::test_deactivate_strategy_flips_bit_back (l1_gas: ~0, l1_data_gas: ~10080,
↳ l2_gas: ~21076620)
[PASS] arcxcore::tests::core_strategy::test_get_strategy_reverts_for_unknown (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~1880192)
[PASS] arcxcore::tests::core_upgrades::test_update_ept_token_class_happy_path (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~2191590)
[PASS] arcxcore::tests::core_orders::test_cancel_order_rejects_already_cancelled (l1_gas: ~0, l1_data_gas: ~11712,
↳ l2_gas: ~35282174)
[PASS] arcxcore::tests::core_upgrades::test_update_st_token_class_happy_path (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~2191590)
[PASS] arcxcore::tests::core_upgrades::test_update_ept_token_class_rejects_zero (l1_gas: ~0, l1_data_gas: ~1248,
↳ l2_gas: ~1828410)
[PASS] arcxcore::tests::core_mint_burn::test_vtoken_strategies_vec_retains_all_children (l1_gas: ~0, l1_data_gas:
↳ ~18720, l2_gas: ~147602641)
[PASS] arcxcore::tests::core_upgrades::test_upgrade_ept_token_rejects_zero_address (l1_gas: ~0, l1_data_gas: ~1248,
↳ l2_gas: ~1902040)
[PASS] arcxcore::tests::core_upgrades::test_upgrade_st_token_rejects_zero_address (l1_gas: ~0, l1_data_gas: ~1248,
↳ l2_gas: ~1902040)
[PASS] arcxcore::tests::core_mint_burn::test_strategy_collateralization_holds_across_mint_burn_cycle (l1_gas: ~0,
↳ l1_data_gas: ~12384, l2_gas: ~215195682)
[PASS] arcxcore::tests::core_upgrades::test_update_st_token_class_rejects_zero (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~1828410)
[PASS] arcxcore::tests::core_upgrades::test_upgrade_ept_token_dispatches_upgrade (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19892412)
[PASS] arcxcore::tests::core_orders::test_create_order_buy_ept_locks_vtoken_collateral_and_persists_order (l1_gas: ~0,
↳ l1_data_gas: ~11808, l2_gas: ~33057050)
[PASS] arcxcore::tests::core_orders::test_create_order_buy_reverts_without_sufficient_unlocked_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~11904, l2_gas: ~35472002)
[PASS] arcxcore::tests::core_orders::test_create_order_buy_st_locks_vtoken_collateral_and_persists_order (l1_gas: ~0,
↳ l1_data_gas: ~12000, l2_gas: ~33057400)
[PASS] arcxcore::tests::core_orders::test_cancel_order_sell_ept_unlocks_ept (l1_gas: ~0, l1_data_gas: ~12768, l2_gas:
↳ ~71745761)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_on_apr_above_cap (l1_gas: ~0, l1_data_gas: ~11136,
↳ l2_gas: ~32289080)
```

```
[PASS] arcxcore::tests::core_orders::test_cancel_order_sell_st_unlocks_st (l1_gas: ~0, l1_data_gas: ~12864, l2_gas: ~71744211)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_on_inactive_strategy (l1_gas: ~0, l1_data_gas: ~11040, l2_gas: ~30452236)
[PASS] arcxcore::tests::core_orders::test_cancel_order_rejects_already_filled (l1_gas: ~0, l1_data_gas: ~16608, l2_gas: ~118572020)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_on_matured_strategy (l1_gas: ~0, l1_data_gas: ~11136, l2_gas: ~31567110)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_on_unknown_strategy (l1_gas: ~0, l1_data_gas: ~11136, l2_gas: ~31319070)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_burn_reverts_if_caller_is_not_core (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~19309702)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_on_zero_apr (l1_gas: ~0, l1_data_gas: ~11136, l2_gas: ~30559948)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_when_balance_exists_but_is_already_locked (l1_gas: ~0, l1_data_gas: ~11904, l2_gas: ~35471902)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_burn_reverts_if_owner_is_zero (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~19312802)
[PASS] arcxcore::tests::core_orders::test_create_order_reverts_when_core_is_paused (l1_gas: ~0, l1_data_gas: ~11232, l2_gas: ~32092330)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_constructor_happy_path_sets_defaults (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~19747084)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_constructor_reverts_if_core_is_zero (l1_gas: ~0, l1_data_gas: ~672, l2_gas: ~1008492)
[PASS]
  arcxcore::tests::ept::points_claims::test_ept_claim_all_points_happy_path_redeems_from_core_and_transfers_to_user (l1_gas: ~0, l1_data_gas: ~15264, l2_gas: ~131357214)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_constructor_reverts_if_point_token_is_zero (l1_gas: ~0, l1_data_gas: ~96, l2_gas: ~516180)
[PASS]
  arcxcore::tests::ept::points_claims::test_ept_claim_all_points_happy_path_keeps_total_claims_within_funded_points (l1_gas: ~0, l1_data_gas: ~16224, l2_gas: ~160394013)
[PASS] arcxcore::tests::bridge_admin::test_deposit_reverts_if_registry_not_set (l1_gas: ~0, l1_data_gas: ~2400, l2_gas: ~6669982)
[PASS] arcxcore::tests::core_orders::test_create_order_sell_ept_locks_ept_and_persists_order (l1_gas: ~0, l1_data_gas: ~12864, l2_gas: ~70150979)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_on_unregistered_vtoken (l1_gas: ~0, l1_data_gas: ~7584, l2_gas: ~47539366)
[PASS] arcxcore::tests::core_orders::test_create_order_sell_ept_reverts_without_sufficient_unlocked_ept (l1_gas: ~0, l1_data_gas: ~12960, l2_gas: ~72561881)
[PASS] arcxcore::tests::ept::points_claims::test_ept_claim_all_points_reverts_if_receiver_is_zero (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~19577042)
[PASS] arcxcore::tests::ept::points_claims::test_ept_get_last_synced_claimable_points_returns_zero_for_zero_account (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~18973122)
[PASS] arcxcore::tests::ept::points_claims::test_ept_claim_all_points_returns_zero_before_vt_finalization (l1_gas: ~0, l1_data_gas: ~12576, l2_gas: ~92341268)
[PASS] arcxcore::tests::ept::points_claims::
test_ept_claim_all_points_happy_path_zero_point_period_does_not_block_later_positive_claim (l1_gas: ~0, l1_data_gas: ~15744, l2_gas: ~123255781)
[PASS] arcxcore::tests::ept::points_claims::
test_ept_get_last_synced_claimable_points_happy_path_applies_claim_fee_to_synced_gross_points (l1_gas: ~0, l1_data_gas: ~15168, l2_gas: ~113954621)
[PASS] arcxcore::tests::point_token_redemption::test_paused_point_token_blocks_redemption (l1_gas: ~0, l1_data_gas: ~7008, l2_gas: ~19010100)
[PASS] arcxcore::tests::ept::points_claims::test_ept_sync_global_credits_happy_path_is_idempotent_without_new_vt_state (l1_gas: ~0, l1_data_gas: ~13824, l2_gas: ~97610124)
[PASS]
  arcxcore::tests::ept::points_claims::test_ept_sync_user_happy_path_advances_last_synced_state_for_nonzero_account (l1_gas: ~0, l1_data_gas: ~12960, l2_gas: ~83892013)
[PASS] arcxcore::tests::point_token_redemption::test_paused_point_token_blocks_transfer_source_update (l1_gas: ~0, l1_data_gas: ~6432, l2_gas: ~12323277)
[PASS] arcxcore::tests::ept::fee_regression::test_ept_claim_uses_core_fee_backing_without_double_treasury_transfer (l1_gas: ~0, l1_data_gas: ~11904, l2_gas: ~68630700)
[PASS] arcxcore::tests::ept::points_claims::test_ept_sync_user_happy_path_zero_account_routes_to_global_state (l1_gas: ~0, l1_data_gas: ~13824, l2_gas: ~91590106)
```

```
[PASS] arcxcore::tests::point_token_redemption::test_paused_point_token_blocks_transferable_update (l1_gas: ~0,
↳ l1_data_gas: ~6432, l2_gas: ~12277887)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_default_non_transferable_blocks_transfer (l1_gas: ~0,
↳ l1_data_gas: ~6528, l2_gas: ~12569401)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_default_non_transferable_blocks_transfer_from (l1_gas:
↳ ~0, l1_data_gas: ~6624, l2_gas: ~13576801)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_defaults_to_non_transferable (l1_gas: ~0, l1_data_gas:
↳ ~6528, l2_gas: ~12258041)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_disable_blocks_redemption (l1_gas: ~0, l1_data_gas:
↳ ~6720, l2_gas: ~17451047)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_final_redeemer_gets_rounding_remainder (l1_gas: ~0,
↳ l1_data_gas: ~6720, l2_gas: ~21278284)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_constructor_reverts_if_vt_is_zero (l1_gas: ~0, l1_data_gas:
↳ ~672, l2_gas: ~1011792)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_redemption_is_proportional_across_two_users (l1_gas:
↳ ~0, l1_data_gas: ~6720, l2_gas: ~21710934)
[PASS] arcxcore::tests::point_token_redemption::test_point_token_underlying_asset_is_immutable_once_set (l1_gas: ~0,
↳ l1_data_gas: ~7872, l2_gas: ~21122822)
[PASS]
↳ arcxcore::tests::point_token_redemption::test_registry_can_pause_and_unpause_point_token_while_registry_is_paused
↳ (l1_gas: ~0, l1_data_gas: ~6432, l2_gas: ~12907497)
[PASS] arcxcore::tests::point_token_redemption::test_registry_can_reenable_redemption_without_new_funding (l1_gas: ~0,
↳ l1_data_gas: ~6624, l2_gas: ~21438480)
[PASS] arcxcore::tests::point_token_redemption::test_registry_enable_redemption_rejects_zero_asset (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~11361224)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_mint_reverts_if_caller_is_not_core (l1_gas: ~0, l1_data_gas:
↳ ~10080, l2_gas: ~19309702)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_mint_reverts_if_recipient_is_zero (l1_gas: ~0, l1_data_gas:
↳ ~10080, l2_gas: ~19312802)
[PASS] arcxcore::tests::point_token_redemption::test_registry_enable_redemption_rejects_zero_funding (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~11503154)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_set_claim_fee_basis_points_happy_path_accepts_100pct (l1_gas:
↳ ~0, l1_data_gas: ~10176, l2_gas: ~20029832)
[PASS] arcxcore::tests::point_token_redemption::test_registry_non_owner_cannot_allow_point_token_transfer_source
↳ (l1_gas: ~0, l1_data_gas: ~6336, l2_gas: ~11366004)
[PASS] arcxcore::tests::point_token_redemption::test_registry_non_owner_cannot_enable_point_token_transfers (l1_gas:
↳ ~0, l1_data_gas: ~6336, l2_gas: ~11353884)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_set_claim_fee_basis_points_reverts_if_caller_is_not_core
↳ (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~19605972)
[PASS] arcxcore::tests::point_token_redemption::test_registry_non_owner_cannot_enable_redemption (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~11490604)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_set_claim_fee_basis_points_reverts_if_fee_exceeds_100pct
↳ (l1_gas: ~0, l1_data_gas: ~10080, l2_gas: ~19618072)
[PASS] arcxcore::tests::point_token_redemption::test_registry_owner_can_allow_point_token_transfer_source (l1_gas: ~0,
↳ l1_data_gas: ~6720, l2_gas: ~14875454)
[PASS] arcxcore::tests::ept::constructor_admin::test_ept_upgrade_reverts_if_caller_is_not_core (l1_gas: ~0,
↳ l1_data_gas: ~10080, l2_gas: ~19287282)
[PASS] arcxcore::tests::point_token_redemption::test_registry_owner_can_enable_point_token_transfers (l1_gas: ~0,
↳ l1_data_gas: ~6720, l2_gas: ~14671504)
[PASS] arcxcore::tests::point_token_redemption::test_registry_owner_can_enable_redemption (l1_gas: ~0, l1_data_gas:
↳ ~6624, l2_gas: ~14308664)
[PASS] arcxcore::tests::point_token_redemption::test_registry_pause_blocks_point_token_redemption (l1_gas: ~0,
↳ l1_data_gas: ~7104, l2_gas: ~19624847)
[PASS] arcxcore::tests::point_token_redemption::test_registry_pause_point_token_still_allows_transfer (l1_gas: ~0,
↳ l1_data_gas: ~6816, l2_gas: ~15904577)
[PASS] arcxcore::tests::ept::core_funding::test_core_redeem_points_happy_path_claims_vt_points_and_funds_ept (l1_gas:
↳ ~0, l1_data_gas: ~14592, l2_gas: ~104903413)
[PASS] arcxcore::tests::ept::core_funding::test_core_redeem_points_happy_path_funds_ept_once_without_double_funding
↳ (l1_gas: ~0, l1_data_gas: ~14592, l2_gas: ~115157578)
[PASS] arcxcore::tests::ept::core_funding::test_ept_claim_fee_happy_path_zero_routes_all_points_to_user (l1_gas: ~0,
↳ l1_data_gas: ~15264, l2_gas: ~114705421)
```

```
[PASS] arcxcore::tests::ept::core_funding::test_ept_claim_fee_happy_path_transfers_net_to_user_and_fee_to_fee_treasury
↳ (l1_gas: ~0, l1_data_gas: ~15552, l2_gas: ~131764624)
[PASS] arcxcore::tests::ept::credit_flows::test_core_mint_st_ept_happy_path_syncs_ept_global_and_recipient (l1_gas: ~0,
↳ l1_data_gas: ~12576, l2_gas: ~76958215)
[PASS] arcxcore::tests::ept::core_funding::
test_ept_sync_global_credits_happy_path_global_earned_points_not_reduced_by_user_claim (l1_gas: ~0, l1_data_gas:
↳ ~15264, l2_gas: ~124317487)
[PASS] arcxcore::tests::ept::credit_flows::test_core_burn_st_ept_happy_path_syncs_global_state_on_burn (l1_gas: ~0,
↳ l1_data_gas: ~13536, l2_gas: ~113026508)
[PASS] arcxcore::tests::ept::credit_flows::
test_core_burn_st_ept_happy_path_preserves_past_entitlement_and_removes_future_balance (l1_gas: ~0, l1_data_gas:
↳ ~15552, l2_gas: ~166481067)
[PASS] arcxcore::tests::point_token_redemption::test_vt_claim_then_redeem_points_for_underlying (l1_gas: ~0,
↳ l1_data_gas: ~9504, l2_gas: ~43817943)
[PASS] arcxcore::tests::ept::credit_flows::test_ept_transfer_blocks_if_amount_exceeds_unlocked_balance (l1_gas: ~0,
↳ l1_data_gas: ~12768, l2_gas: ~101168154)
[PASS] arcxcore::tests::ept::credit_flows::test_ept_transfer_happy_path_allows_unlocked_remainder (l1_gas: ~0,
↳ l1_data_gas: ~13152, l2_gas: ~101990794)
[PASS] arcxcore::tests::point_token_redemption::test_vt_submit_points_rejects_when_redemption_enabled (l1_gas: ~0,
↳ l1_data_gas: ~8352, l2_gas: ~33256380)
[PASS] arcxcore::tests::registry_contract::test_add_exchange_happy_path_sets_exchange_fields (l1_gas: ~0, l1_data_gas:
↳ ~1824, l2_gas: ~4152143)
[PASS] arcxcore::tests::ept::credit_flows::test_ept_transfer_happy_path_self_transfer_does_not_double_sync (l1_gas: ~0,
↳ l1_data_gas: ~13344, l2_gas: ~102131206)
[PASS] arcxcore::tests::registry_contract::test_add_exchange_rejects_duplicate_exchange_id (l1_gas: ~0, l1_data_gas:
↳ ~1824, l2_gas: ~2881481)
[PASS] arcxcore::tests::registry_contract::test_bridge_ownership_transfer_requires_acceptance (l1_gas: ~0, l1_data_gas:
↳ ~192, l2_gas: ~2601360)
[PASS] arcxcore::tests::core_orders::test_create_order_sell_st_locks_st (l1_gas: ~0, l1_data_gas: ~12960, l2_gas:
↳ ~69930119)
[PASS] arcxcore::tests::ept::credit_flows::test_ept_transfer_happy_path_syncs_sender_and_receiver_before_balance_change
↳ (l1_gas: ~0, l1_data_gas: ~15840, l2_gas: ~151468608)
[PASS] arcxcore::tests::registry_contract::test_bridge_queries_registry_bidirectional_mapping (l1_gas: ~0, l1_data_gas:
↳ ~6528, l2_gas: ~10929438)
[PASS] arcxcore::tests::registry_contract::test_registry_accepts_zero_vt_apr (l1_gas: ~0, l1_data_gas: ~6336, l2_gas:
↳ ~10415858)
[PASS] arcxcore::tests::registry_contract::test_registry_mint_exchange_points_rejects_zero_recipient (l1_gas: ~0,
↳ l1_data_gas: ~6528, l2_gas: ~10716618)
[PASS] arcxcore::tests::registry_contract::test_registry_mint_exchange_points_rejects_unregistered_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~6528, l2_gas: ~10823148)
[PASS] arcxcore::tests::registry_contract::test_registry_multiple_vts_share_point_token_and_can_mint_points (l1_gas:
↳ ~0, l1_data_gas: ~9984, l2_gas: ~19534395)
[PASS] arcxcore::tests::ept::points_claims::
test_ept_claim_all_points_happy_path_advances_claim_state_and_second_claim_returns_zero (l1_gas: ~0, l1_data_gas:
↳ ~15264, l2_gas: ~148420047)
[PASS] arcxcore::tests::registry_contract::test_registry_pause_blocks_add_exchange (l1_gas: ~0, l1_data_gas: ~576,
↳ l2_gas: ~2660306)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_withdrawal_nft_name (l1_gas: ~0, l1_data_gas:
↳ ~3264, l2_gas: ~7177483)
[PASS] arcxcore::tests::registry_contract::test_registry_pause_blocks_add_vt (l1_gas: ~0, l1_data_gas: ~3360, l2_gas:
↳ ~8462793)
[PASS] arcxcore::tests::ept::points_claims::test_ept_claim_all_points_happy_path_can_send_to_different_receiver
↳ (l1_gas: ~0, l1_data_gas: ~15264, l2_gas: ~113665371)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_withdrawal_nft_symbol (l1_gas: ~0, l1_data_gas:
↳ ~3264, l2_gas: ~7185433)
[PASS] arcxcore::tests::registry_contract::test_registry_pause_blocks_mint_points (l1_gas: ~0, l1_data_gas: ~6624,
↳ l2_gas: ~11878881)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_bridge_proxy (l1_gas: ~0, l1_data_gas: ~3264,
↳ l2_gas: ~7169643)
[PASS] arcxcore::tests::registry_contract::test_registry_pause_blocks_point_token_transferable_update (l1_gas: ~0,
↳ l1_data_gas: ~1920, l2_gas: ~4047191)
[PASS] arcxcore::tests::registry_contract::test_registry_pause_blocks_point_token_transfer_source_update (l1_gas: ~0,
↳ l1_data_gas: ~1920, l2_gas: ~4059211)
```

```
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_exchange_id (l1_gas: ~0, l1_data_gas: ~3264,
↳ l2_gas: ~6890990)
[PASS] arcxcore::tests::ept::points_claims::
test_ept_claim_all_points_happy_path_allows_former_holder_to_claim_pre_sale_entitlement (l1_gas: ~0, l1_data_gas:
↳ ~16032, l2_gas: ~182460074)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_first_period_boundary (l1_gas: ~0,
↳ l1_data_gas: ~3264, l2_gas: ~7169433)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_duplicate_market (l1_gas: ~0, l1_data_gas: ~6528,
↳ l2_gas: ~11700461)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_market (l1_gas: ~0, l1_data_gas: ~3264,
↳ l2_gas: ~7154473)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_owner (l1_gas: ~0, l1_data_gas: ~3264, l2_gas:
↳ ~7160043)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_operator (l1_gas: ~0, l1_data_gas: ~3264,
↳ l2_gas: ~7163243)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_period_duration (l1_gas: ~0, l1_data_gas:
↳ ~3264, l2_gas: ~7167333)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_salt (l1_gas: ~0, l1_data_gas: ~3264, l2_gas:
↳ ~7133333)
[PASS] arcxcore::tests::registry_contract::test_registry_unknown_vtoken_is_not_registered (l1_gas: ~0, l1_data_gas:
↳ ~6528, l2_gas: ~10421858)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_zero_vt_treasury (l1_gas: ~0, l1_data_gas: ~3264,
↳ l2_gas: ~7166443)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_exchange_name (l1_gas: ~0, l1_data_gas: ~1824,
↳ l2_gas: ~3129788)
[PASS] arcxcore::tests::registry_contract::test_registry_unpause_restores_add_vt (l1_gas: ~0, l1_data_gas: ~6432,
↳ l2_gas: ~11944078)
[PASS] arcxcore::tests::registry_contract::test_registry_update_class_hashes_and_upgrade_exchange_point_token (l1_gas:
↳ ~0, l1_data_gas: ~1824, l2_gas: ~5282024)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_point_token_name (l1_gas: ~0, l1_data_gas:
↳ ~1824, l2_gas: ~3137738)
[PASS] arcxcore::tests::registry_contract::test_vt_owner_can_manage_config_without_acceptance (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~11719858)
[PASS] arcxcore::tests::core_orders::test_create_order_sell_st_reverts_without_sufficient_unlocked_st (l1_gas: ~0,
↳ l1_data_gas: ~13056, l2_gas: ~72559581)
[PASS] arcxcore::tests::registry_contract::test_registry_vt_creation_wires_all_vt_config (l1_gas: ~0, l1_data_gas:
↳ ~7008, l2_gas: ~22085258)
[PASS] arcxcore::tests::user_credit_component::test_burn_happy_path_accrues_only_before_burn_ci (l1_gas: ~0,
↳ l1_data_gas: ~2592, l2_gas: ~15372132)
[PASS] arcxcore::tests::core_orders::test_next_order_id_is_monotonic (l1_gas: ~0, l1_data_gas: ~12960, l2_gas:
↳ ~37244584)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_point_token_symbol (l1_gas: ~0, l1_data_gas:
↳ ~1824, l2_gas: ~3145688)
[PASS] arcxcore::tests::user_credit_component::
test_claim_points_happy_path_consumes_pending_allocation_after_boundary_settlement (l1_gas: ~0, l1_data_gas: ~2688,
↳ l2_gas: ~16752222)
[PASS] arcxcore::tests::user_credit_component::
test_claim_points_happy_path_tracks_total_claimed_points_across_multiple_users (l1_gas: ~0, l1_data_gas: ~3456, l2_gas:
↳ ~24678432)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_vt_name (l1_gas: ~0, l1_data_gas: ~3264, l2_gas:
↳ ~7141383)
[PASS] arcxcore::tests::user_credit_component::
test_claim_points_returns_zero_for_global_user_without_incrementing_user_point_totals (l1_gas: ~0, l1_data_gas: ~2400,
↳ l2_gas: ~10290702)
[PASS] arcxcore::tests::registry_contract::test_registry_rejects_empty_vt_symbol (l1_gas: ~0, l1_data_gas: ~3264,
↳ l2_gas: ~7149333)
[PASS] arcxcore::tests::user_credit_component::test_claim_points_returns_zero_when_nothing_allocated (l1_gas: ~0,
↳ l1_data_gas: ~1728, l2_gas: ~7072632)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_works_with_zero_ppc_points (l1_gas: ~0,
↳ l1_data_gas: ~2208, l2_gas: ~8796622)
[PASS] arcxcore::tests::core_orders::test_update_limit_apr_happy_path (l1_gas: ~0, l1_data_gas: ~11808, l2_gas:
↳ ~34115070)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_closed_period_ci_goes_backwards (l1_gas: ~0,
↳ l1_data_gas: ~2016, l2_gas: ~8013342)
```

```
[PASS] arcxcore::tests::core_orders::test_update_limit_apr_only_self (l1_gas: ~0, l1_data_gas: ~11808, l2_gas:
↳ ~32984910)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_current_ci_goes_backwards (l1_gas: ~0,
↳ l1_data_gas: ~1920, l2_gas: ~9749832)
[PASS] arcxcore::tests::core_orders::test_sell_ept_order_locked_ept_blocks_transfer_above_unlocked_remainder (l1_gas:
↳ ~0, l1_data_gas: ~12864, l2_gas: ~93343428)
[PASS] arcxcore::tests::core_orders::test_sell_st_order_locked_st_blocks_transfer_above_unlocked_remainder (l1_gas: ~0,
↳ l1_data_gas: ~12960, l2_gas: ~70466139)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_current_live_period_is_before_finalized_period
↳ (l1_gas: ~0, l1_data_gas: ~2496, l2_gas: ~9981522)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_initial_checkpoint_is_stale (l1_gas: ~0,
↳ l1_data_gas: ~960, l2_gas: ~2985052)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_constructor_sets_owner_roles_core_and_pause_state (l1_gas:
↳ ~0, l1_data_gas: ~9792, l2_gas: ~14939847)
[PASS] arcxcore::tests::core_orders::test_update_limit_apr_rejects_zero (l1_gas: ~0, l1_data_gas: ~11808, l2_gas:
↳ ~32983880)
[PASS] arcxcore::tests::user_credit_component::test_credit_math_happy_path_staggered_entry (l1_gas: ~0, l1_data_gas:
↳ ~3456, l2_gas: ~20721752)
[PASS] arcxcore::tests::user_credit_component::test_credit_math_happy_path_supply_burn (l1_gas: ~0, l1_data_gas: ~3360,
↳ l2_gas: ~22615842)
[PASS] arcxcore::tests::user_credit_component::test_credit_math_happy_path_two_users_same_interval (l1_gas: ~0,
↳ l1_data_gas: ~3456, l2_gas: ~19859532)
[PASS] arcxcore::tests::core_orders::test_update_limit_apr_respects_cap (l1_gas: ~0, l1_data_gas: ~11808, l2_gas:
↳ ~34196750)
[PASS] arcxcore::tests::user_credit_component::test_mint_happy_path_accrues_only_after_mint_ci (l1_gas: ~0,
↳ l1_data_gas: ~2784, l2_gas: ~13476542)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_depositor_can_pause_and_unpause_investment (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15822137)
[PASS] arcxcore::tests::user_credit_component::test_mint_happy_path_accrues_piecewise_with_multiple_mints (l1_gas: ~0,
↳ l1_data_gas: ~2784, l2_gas: ~21835362)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_initial_checkpoint_period_is_zero (l1_gas: ~0,
↳ l1_data_gas: ~864, l2_gas: ~2980472)
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_happy_path (l1_gas: ~0, l1_data_gas: ~12384, l2_gas:
↳ ~75730749)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_last_nav_update_time_is_stale (l1_gas: ~0,
↳ l1_data_gas: ~960, l2_gas: ~2984592)
[PASS] arcxcore::tests::user_credit_component::test_mint_happy_path_splits_piecewise_credits_between_users (l1_gas: ~0,
↳ l1_data_gas: ~3456, l2_gas: ~24742792)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_reverts_if_points_backing_is_insufficient (l1_gas: ~0,
↳ l1_data_gas: ~2112, l2_gas: ~8132442)
[PASS] arcxcore::tests::user_credit_component::test_transfer_happy_path_initializes_receiver_at_transfer_ci (l1_gas:
↳ ~0, l1_data_gas: ~3072, l2_gas: ~16892132)
[PASS] arcxcore::tests::user_credit_component::test_sync_global_allows_initial_ci_to_be_nonzero (l1_gas: ~0,
↳ l1_data_gas: ~1440, l2_gas: ~3314322)
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_reverts_before_maturity (l1_gas: ~0, l1_data_gas: ~12096,
↳ l2_gas: ~67491819)
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_reverts_on_unknown_strategy (l1_gas: ~0, l1_data_gas:
↳ ~10176, l2_gas: ~19935124)
[PASS] arcxcore::tests::user_credit_component::test_transfer_happy_path_preserves_existing_receiver_history (l1_gas:
↳ ~0, l1_data_gas: ~3072, l2_gas: ~20351002)
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_reverts_on_zero_amount (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19675532)
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_reverts_on_zero_receiver (l1_gas: ~0, l1_data_gas: ~10176,
↳ l2_gas: ~19697362)
[PASS] arcxcore::tests::user_credit_component::test_transfer_happy_path_works_at_period_boundary (l1_gas: ~0,
↳ l1_data_gas: ~3264, l2_gas: ~18518532)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_depositor_cannot_unpause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~15612037)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_operator_can_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~15218697)
```



```
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_reverts_when_amount_exceeds_available_balance (l1_gas: ~0,
↳ l1_data_gas: ~12096, l2_gas: ~68498859)
[PASS] arcxcore::tests::core_redeem::test_redeem_matured_st_reverts_when_st_is_locked_on_order (l1_gas: ~0,
↳ l1_data_gas: ~13056, l2_gas: ~71981721)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_moves_usdc_from_depositor_to_vault (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~18525327)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_operator_cannot_pause_investment (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~14770977)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_operator_cannot_unpause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~15118097)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_rejects_failed_transfer_from (l1_gas: ~0, l1_data_gas:
↳ ~20544, l2_gas: ~31616744)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_rejects_investment_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~16002287)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_owner_can_pause_and_unpause (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15501197)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_rejects_local_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~15958517)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_owner_can_upgrade_contract (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~14977197)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_rejects_non_depositor (l1_gas: ~0, l1_data_gas: ~9792,
↳ l2_gas: ~15167157)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_rejects_registry_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~16076347)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_owner_can_upgrade_while_investment_paused (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~15712857)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_owner_can_upgrade_while_locally_paused (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~15811477)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_mints_fewer_shares_after_nav_growth (l1_gas:
↳ ~0, l1_data_gas: ~12096, l2_gas: ~42794439)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_pause_investment_rejects_double_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~14965577)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_rejects_investment_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16200387)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_unpause_investment_rejects_when_not_paused (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15058127)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_unpause_investment_rejects_when_vault_is_paused (l1_gas:
↳ ~0, l1_data_gas: ~9984, l2_gas: ~16484477)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_rejects_local_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~16156617)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_rejects_outsider (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15397187)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_upgrade_contract_rejects_non_owner (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~14714547)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_rejects_registry_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16274447)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_upgrade_contract_rejects_zero_class_hash (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~14716257)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_order_rejects_local_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~16577017)
[PASS] arcxcore::tests::user_credit_component::test_sync_global_happy_path_ignores_user_claims (l1_gas: ~0,
↳ l1_data_gas: ~2976, l2_gas: ~14969772)
[PASS] arcxcore::tests::vault::orders::test_vault_cancel_order_forwards_to_core_and_marks_order_cancelled (l1_gas: ~0,
↳ l1_data_gas: ~11616, l2_gas: ~32572867)
[PASS] arcxcore::tests::user_credit_component::test_sync_global_happy_path_initializes_new_global_state (l1_gas: ~0,
↳ l1_data_gas: ~1440, l2_gas: ~4173082)
[PASS] arcxcore::tests::vault::orders::test_vault_cancel_order_rejects_local_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~15824857)
[PASS]
↳ arcxcore::tests::user_credit_component::test_sync_global_happy_path_zero_account_catches_up_existing_global_state
↳ (l1_gas: ~0, l1_data_gas: ~3360, l2_gas: ~13881212)
```

```
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_rejects_unregistered_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~10272, l2_gas: ~16316647)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_rejects_zero_vtoken (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15308867)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_order_rejects_outsider (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15817487)
[PASS]
↳ arcxcore::tests::user_credit_component::test_sync_global_ignores_initial_current_period_when_checkpoint_is_period_one
↳ (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~3314122)
[PASS] arcxcore::tests::vault::orders::test_vault_cancel_order_rejects_outsider (l1_gas: ~0, l1_data_gas: ~9792,
↳ l2_gas: ~15065327)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_on_wrong_action_type (l1_gas: ~0,
↳ l1_data_gas: ~7584, l2_gas: ~35569626)
[PASS] arcxcore::tests::user_credit_component::test_sync_global_reverts_if_initial_checkpoint_period_is_zero (l1_gas:
↳ ~0, l1_data_gas: ~864, l2_gas: ~2971272)
[PASS] arcxcore::tests::vault::orders::test_vault_cancel_order_rejects_registry_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~15942687)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_deposit_to_vtoken_uses_direct_vtoken_address (l1_gas: ~0,
↳ l1_data_gas: ~10656, l2_gas: ~27710703)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_buy_locks_vtoken_collateral (l1_gas: ~0, l1_data_gas:
↳ ~11520, l2_gas: ~31503087)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_withdraw_idle_rejects_insufficient_balance (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15314837)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_buy_rejects_investment_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~16270237)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_withdraw_idle_rejects_local_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~15907467)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_rejects_invalid_strategy (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15835499)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_withdraw_idle_rejects_non_depositor (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15116207)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_rejects_local_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~16226467)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_accrues_multiple_periods (l1_gas: ~0,
↳ l1_data_gas: ~2400, l2_gas: ~9972422)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_withdraw_idle_rejects_registry_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~16025297)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_rejects_outsider (l1_gas: ~0, l1_data_gas: ~9792,
↳ l2_gas: ~15466937)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_accrues_single_live_period (l1_gas: ~0,
↳ l1_data_gas: ~2016, l2_gas: ~7761402)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_reverts_on_zero_delta (l1_gas: ~0, l1_data_gas:
↳ ~7488, l2_gas: ~28229046)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_accrues_162_periods (l1_gas: ~0, l1_data_gas:
↳ ~48384, l2_gas: ~145414052)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_catches_up_existing_user_state (l1_gas: ~0,
↳ l1_data_gas: ~3360, l2_gas: ~17669562)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_rejects_registry_pause (l1_gas: ~0, l1_data_gas: ~9888,
↳ l2_gas: ~16344297)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_withdraw_idle_rejects_zero_token (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15104387)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_rejects_unregistered_strategy_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16656281)
[PASS] arcxcore::tests::vault::asset_flows::test_vault_withdraw_idle_still_accepts_generic_token_address (l1_gas: ~0,
↳ l1_data_gas: ~10848, l2_gas: ~34355513)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_constructor_rejects_zero_core (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~14620387)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_constructor_rejects_zero_depositor (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~14607487)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_surplus_routes_excess_to_treasury (l1_gas: ~0,
↳ l1_data_gas: ~7680, l2_gas: ~51791406)
```



```
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_constructor_rejects_zero_operator (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~14610787)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_sell_ept_locks_ept_collateral (l1_gas: ~0, l1_data_gas:
↳ ~11232, l2_gas: ~35494473)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_sell_is_allowed_during_investment_pause (l1_gas: ~0,
↳ l1_data_gas: ~11328, l2_gas: ~36229933)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_handles_period_boundary (l1_gas: ~0,
↳ l1_data_gas: ~2112, l2_gas: ~8891712)
[PASS] arcxcore::tests::bridge_receive_return::test_receive_and_return_uses_balance_delta_not_hook_amount (l1_gas: ~0,
↳ l1_data_gas: ~7680, l2_gas: ~51602036)
[PASS] arcxcore::tests::vault::orders::test_vault_submit_order_sell_st_locks_strategy_tokens (l1_gas: ~0, l1_data_gas:
↳ ~10944, l2_gas: ~19663601)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_constructor_rejects_zero_owner (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~14613987)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_initializes_new_user_state (l1_gas: ~0,
↳ l1_data_gas: ~1440, l2_gas: ~3702702)
[PASS] arcxcore::tests::bridge_withdraw_request::test_request_withdraw_caller_must_be_registered_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~7488, l2_gas: ~13718368)
[PASS] arcxcore::tests::vault::constructor_admin::test_vault_constructor_rejects_zero_usdc (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~14617187)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_is_idempotent_without_ci_advance (l1_gas: ~0,
↳ l1_data_gas: ~2496, l2_gas: ~12324192)
[PASS] arcxcore::tests::bridge_withdraw_request::test_request_withdraw_encodes_message_body (l1_gas: ~0, l1_data_gas:
↳ ~8256, l2_gas: ~33205458)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_order_rejects_registry_pause (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~16694847)
[PASS] arcxcore::tests::bridge_withdraw_request::test_request_withdraw_happy_path (l1_gas: ~0, l1_data_gas: ~8352,
↳ l2_gas: ~21703657)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_order_rejects_sell_orders (l1_gas: ~0, l1_data_gas:
↳ ~9888, l2_gas: ~19020447)
[PASS] arcxcore::tests::user_credit_component::
test_sync_user_happy_path_leaves_live_period_unsettled_then_settles_stored_credits (l1_gas: ~0, l1_data_gas: ~2880,
↳ l2_gas: ~13788112)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_ept_points_rejects_local_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16034767)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_ept_points_rejects_outsider (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15275237)
[PASS] arcxcore::tests::bridge_withdraw_request::test_request_withdraw_returns_monotonic_ids (l1_gas: ~0, l1_data_gas:
↳ ~8352, l2_gas: ~36639125)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_ept_points_rejects_registry_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16152597)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_buy_order_resolves_vtoken_from_strategy (l1_gas: ~0,
↳ l1_data_gas: ~11520, l2_gas: ~31560305)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_works_with_zero_balance (l1_gas: ~0,
↳ l1_data_gas: ~1728, l2_gas: ~7005982)
[PASS] arcxcore::tests::bridge_withdraw_request::test_request_withdraw_reverts_on_zero_amount (l1_gas: ~0, l1_data_gas:
↳ ~7488, l2_gas: ~13536788)
[PASS] arcxcore::tests::user_credit_component::test_sync_user_happy_path_settles_108_finalized_periods (l1_gas: ~0,
↳ l1_data_gas: ~33312, l2_gas: ~125792762)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_ept_points_rejects_unknown_strategy_identifier
↳ (l1_gas: ~0, l1_data_gas: ~9792, l2_gas: ~15688019)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_buy_order_uses_nav_grown_preview_mint (l1_gas: ~0,
↳ l1_data_gas: ~12864, l2_gas: ~48099847)
[PASS] arcxcore::tests::vault::orders::test_vault_wrap_and_place_order_rejects_investment_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16620787)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_ept_points_rejects_zero_ept_token_strategy (l1_gas:
↳ ~0, l1_data_gas: ~9792, l2_gas: ~16182351)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_vt_points_rejects_local_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16021467)
[PASS] arcxcore::tests::core_admin::test_activate_strategy_only_owner (l1_gas: ~0, l1_data_gas: ~10080, l2_gas:
↳ ~18929762)
```

```
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_vt_points_rejects_outsider (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15262037)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_cancel_order_rejects_reentrant_core_callback (l1_gas: ~0,
↳ l1_data_gas: ~12672, l2_gas: ~20138421)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_ept_points_supports_strategy_identifier (l1_gas:
↳ ~0, l1_data_gas: ~13536, l2_gas: ~74545858)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_claim_ept_points_rejects_reentrant_vtoken_callback (l1_gas: ~0,
↳ l1_data_gas: ~11616, l2_gas: ~19693211)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_withdraw_batch_delay_requires_admin (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~11271831)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_claim_vt_points_rejects_reentrant_vtoken_callback (l1_gas: ~0,
↳ l1_data_gas: ~11616, l2_gas: ~19469709)
[PASS] arcxcore::tests::core_admin::test_add_strategy_only_owner (l1_gas: ~0, l1_data_gas: ~7104, l2_gas: ~12920453)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_deposit_rejects_reentrant_erc20_callback (l1_gas: ~0,
↳ l1_data_gas: ~11232, l2_gas: ~19109747)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_vt_points_rejects_registry_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16139297)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_vt_points_rejects_unregistered_vtoken_identifier
↳ (l1_gas: ~0, l1_data_gas: ~10272, l2_gas: ~16224717)
[PASS] arcxcore::tests::core_admin::test_constructor_match_nonce_starts_at_0 (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~1512970)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_vt_points_rejects_zero_vtoken_identifier (l1_gas:
↳ ~0, l1_data_gas: ~9792, l2_gas: ~15216937)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_redeem_vtoken_rejects_reentrant_vtoken_callback (l1_gas: ~0,
↳ l1_data_gas: ~11616, l2_gas: ~19781659)
[PASS] arcxcore::tests::core_admin::test_constructor_next_order_id_nonzero_so_get_order_zero_panics (l1_gas: ~0,
↳ l1_data_gas: ~1248, l2_gas: ~1923600)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_redeem_vtoken_rejects_local_pause (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~16111157)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_claim_vt_points_supports_vtoken_address (l1_gas: ~0,
↳ l1_data_gas: ~12864, l2_gas: ~48215416)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_deposit_to_vtoken_rejects_reentrant_vtoken_callback (l1_gas: ~0,
↳ l1_data_gas: ~11712, l2_gas: ~23771839)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_redeem_vtoken_rejects_outsider (l1_gas: ~0, l1_data_gas:
↳ ~9792, l2_gas: ~15351727)
[PASS] arcxcore::tests::core_admin::test_constructor_wires_owner_and_class_hashes (l1_gas: ~0, l1_data_gas: ~1248,
↳ l2_gas: ~2344960)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_submit_order_rejects_reentrant_core_callback (l1_gas: ~0,
↳ l1_data_gas: ~12096, l2_gas: ~20195913)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_trigger_withdrawal_request_rejects_reentrant_vtoken_callback
↳ (l1_gas: ~0, l1_data_gas: ~11616, l2_gas: ~19533689)
[PASS] arcxcore::tests::core_admin::test_deactivate_strategy_only_owner (l1_gas: ~0, l1_data_gas: ~10176, l2_gas:
↳ ~19793936)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_withdraw_idle_rejects_reentrant_erc20_callback (l1_gas: ~0,
↳ l1_data_gas: ~11232, l2_gas: ~18453807)
[PASS] arcxcore::tests::core_admin::test_get_version_returns_v0 (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1532680)
[PASS] arcxcore::tests::vault::reentrancy::test_vault_wrap_and_place_order_rejects_reentrant_vtoken_callback (l1_gas:
↳ ~0, l1_data_gas: ~11712, l2_gas: ~24491711)
[PASS] arcxcore::tests::core_admin::test_pause_and_unpause_strategy_token_toggle_child_state (l1_gas: ~0, l1_data_gas:
↳ ~10176, l2_gas: ~20997542)
[PASS] arcxcore::tests::core_admin::test_pause_and_unpause_toggle_core_state (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~3192360)
[PASS] arcxcore::tests::core_admin::test_pause_only_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1635870)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_allows_first_boundary_exactly_one_full_period_away
↳ (l1_gas: ~0, l1_data_gas: ~4224, l2_gas: ~6081429)
[PASS] arcxcore::tests::core_admin::test_pause_st_ept_only_owner (l1_gas: ~0, l1_data_gas: ~10176, l2_gas: ~18963972)
```

```
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_first_boundary_before_now (l1_gas: ~0,
↳ l1_data_gas: ~2592, l2_gas: ~5948559)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_performance_fee_above_100pct (l1_gas: ~0,
↳ l1_data_gas: ~2592, l2_gas: ~5705109)
[PASS]
↳ arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_when_first_boundary_is_more_than_one_period_away
  (l1_gas: ~0, l1_data_gas: ~2592, l2_gas: ~5948549)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_zero_admin (l1_gas: ~0, l1_data_gas: ~2592,
↳ l2_gas: ~5697259)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_zero_period_duration (l1_gas: ~0,
↳ l1_data_gas: ~2592, l2_gas: ~5697359)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_zero_point_token (l1_gas: ~0, l1_data_gas:
↳ ~2016, l2_gas: ~5198247)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_reverts_zero_withdrawal_nft (l1_gas: ~0,
↳ l1_data_gas: ~1920, l2_gas: ~5131816)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_constructor_sets_nav_ci_period_admins (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~13480651)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_get_version_returns_v0 (l1_gas: ~0, l1_data_gas: ~6336, l2_gas:
↳ ~10690061)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_withdraw_batch_delay_updates_value (l1_gas: ~0, l1_data_gas:
↳ ~6432, l2_gas: ~11745291)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_upgrade_admin_only (l1_gas: ~0, l1_data_gas: ~6336, l2_gas:
↳ ~11011981)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_burned_balance_stops_earning_future_credits (l1_gas: ~0, l1_data_gas:
↳ ~8352, l2_gas: ~41552965)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_balance_changes_in_same_period_do_not_overallocate_points (l1_gas:
↳ ~0, l1_data_gas: ~10368, l2_gas: ~75059442)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_deposit_after_ci_growth_only_earns_future_credits (l1_gas: ~0,
↳ l1_data_gas: ~9888, l2_gas: ~53466196)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_deposit_after_nav_growth_gets_fewer_shares (l1_gas: ~0, l1_data_gas:
↳ ~7008, l2_gas: ~29268031)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_deposit_records_daily_inflow (l1_gas: ~0, l1_data_gas: ~7584, l2_gas:
↳ ~23344387)
[PASS] arcxcore::tests::vt::credit_flows::
test_vt_deposit_same_timestamp_after_initial_deposit_mints_adjacent_shares_with_virtual_offset_rounding (l1_gas: ~0,
↳ l1_data_gas: ~8928, l2_gas: ~44965689)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_deposit_same_timestamp_uses_same_entry_credit_index (l1_gas: ~0,
↳ l1_data_gas: ~8928, l2_gas: ~46358689)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_deposit_with_referral_deposits_and_emits_referral_code (l1_gas: ~0,
↳ l1_data_gas: ~7200, l2_gas: ~21503727)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_deposit_updates_nav_before_mint (l1_gas: ~0, l1_data_gas: ~7584,
↳ l2_gas: ~23147927)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_get_credit_index_returns_integrated_nav_growth (l1_gas: ~0,
↳ l1_data_gas: ~7200, l2_gas: ~21686137)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_transfer_reverts_when_vt_is_paused (l1_gas: ~0, l1_data_gas: ~7680,
↳ l2_gas: ~33925337)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_request_withdrawal_preserves_credits_until_burn (l1_gas: ~0,
↳ l1_data_gas: ~12096, l2_gas: ~74841356)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_checkpoint_with_zero_supply_sets_end_ci_but_zero_credits (l1_gas: ~0,
↳ l1_data_gas: ~6528, l2_gas: ~14886141)
[PASS] arcxcore::tests::vt::credit_flows::test_vt_transfer_syncs_sender_and_receiver_before_balance_change (l1_gas: ~0,
↳ l1_data_gas: ~8448, l2_gas: ~31635887)
[PASS] arcxcore::tests::vt::credit_flows::
test_vt_transfer_preserves_sender_past_entitlement_and_receiver_future_entitlement (l1_gas: ~0, l1_data_gas: ~9984,
↳ l2_gas: ~50730030)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_deposit_at_exact_boundary_gets_no_previous_period_credits (l1_gas: ~0,
↳ l1_data_gas: ~8928, l2_gas: ~36167010)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_deposit_does_not_decrease_credit_index (l1_gas: ~0, l1_data_gas:
↳ ~7584, l2_gas: ~24092377)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_deposit_one_second_before_boundary_earns_previous_period_credits
  (l1_gas: ~0, l1_data_gas: ~8064, l2_gas: ~27166197)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_get_current_credit_period_uses_first_boundary_and_duration (l1_gas:
↳ ~0, l1_data_gas: ~6336, l2_gas: ~12792931)
```

```
[PASS] arcxcore::tests::vt::nav_periods::test_vt_get_nav_returns_scale_at_deploy (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~10791111)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_get_nav_rate_increases_after_update_nav_reanchors_nav (l1_gas: ~0,
↳ l1_data_gas: ~7776, l2_gas: ~25345027)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_get_nav_returns_linear_apr_growth_after_elapsed_time (l1_gas: ~0,
↳ l1_data_gas: ~7200, l2_gas: ~21497897)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_nav_resets_to_scale_when_total_supply_returns_to_zero (l1_gas: ~0,
↳ l1_data_gas: ~9696, l2_gas: ~38736347)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_applies_nav_performance_fee_to_nav_and_accrued_fees
↳ (l1_gas: ~0, l1_data_gas: ~7968, l2_gas: ~24711437)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_closes_multiple_crossed_periods (l1_gas: ~0, l1_data_gas:
↳ ~8352, l2_gas: ~29712717)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_checkpoints_only_crossed_period_boundaries (l1_gas: ~0,
↳ l1_data_gas: ~8160, l2_gas: ~29267177)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_closes_one_crossed_period (l1_gas: ~0, l1_data_gas: ~7968,
↳ l2_gas: ~26161367)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_catches_up_many_crossed_periods (l1_gas: ~0, l1_data_gas:
↳ ~15936, l2_gas: ~85222665)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_does_not_mutate_closed_period_credits (l1_gas: ~0,
↳ l1_data_gas: ~8352, l2_gas: ~31232347)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_exactly_at_boundary_closes_period (l1_gas: ~0, l1_data_gas:
↳ ~7872, l2_gas: ~25732507)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_inside_period_does_not_close_completed_period (l1_gas: ~0,
↳ l1_data_gas: ~7776, l2_gas: ~25249607)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_persists_nav_ci_and_checkpoint_time (l1_gas: ~0,
↳ l1_data_gas: ~7776, l2_gas: ~24648067)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_resets_live_period_credits_after_boundary (l1_gas: ~0,
↳ l1_data_gas: ~7968, l2_gas: ~26675227)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_update_nav_same_timestamp_is_idempotent (l1_gas: ~0, l1_data_gas:
↳ ~7776, l2_gas: ~26291707)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_zero_supply_closed_periods_do_not_leak_credits_to_later_depositor
↳ (l1_gas: ~0, l1_data_gas: ~8256, l2_gas: ~32984747)
[PASS] arcxcore::tests::vt::nav_periods::test_vt_zero_apr_keeps_nav_constant_but_ci_grows_linearly (l1_gas: ~0,
↳ l1_data_gas: ~6528, l2_gas: ~25250305)
[PASS] arcxcore::tests::vt::points_claims::test_vt_claim_all_points_can_send_to_different_receiver (l1_gas: ~0,
↳ l1_data_gas: ~9216, l2_gas: ~37824410)
[PASS] arcxcore::tests::vt::points_claims::test_vt_claim_all_points_reverts_zero_receiver (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~16738561)
[PASS] arcxcore::tests::vt::points_claims::test_vt_claim_all_points_advances_claim_state_and_second_claim_returns_zero
↳ (l1_gas: ~0, l1_data_gas: ~9216, l2_gas: ~46193400)
[PASS] arcxcore::tests::vt::points_claims::test_vt_claim_all_points_syncs_user_and_transfers_points (l1_gas: ~0,
↳ l1_data_gas: ~9216, l2_gas: ~37624280)
[PASS] arcxcore::tests::vt::points_claims::test_vt_claim_before_finalization_does_not_burn_later_claim (l1_gas: ~0,
↳ l1_data_gas: ~9216, l2_gas: ~41611080)
[PASS] arcxcore::tests::vt::points_claims::test_vt_multiple_user_claims_do_not_exceed_net_allocated_points (l1_gas: ~0,
↳ l1_data_gas: ~11808, l2_gas: ~75270402)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_period_points_rejects_live_period (l1_gas: ~0, l1_data_gas:
↳ ~7872, l2_gas: ~29610737)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_period_points_rejects_period_zero (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~13893711)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_period_points_rejects_repeated_finalized_period (l1_gas: ~0,
↳ l1_data_gas: ~6624, l2_gas: ~18081391)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_period_points_records_net_accounting_and_mints_gross_to_vault
↳ (l1_gas: ~0, l1_data_gas: ~8928, l2_gas: ~33199220)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_period_points_rejects_skipped_period (l1_gas: ~0,
↳ l1_data_gas: ~7584, l2_gas: ~25988247)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_positive_points_requires_period_credits (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~14959421)
[PASS] arcxcore::tests::vt::points_claims::test_vt_submit_period_points_requires_admin (l1_gas: ~0, l1_data_gas: ~7584,
↳ l2_gas: ~25690287)
```

```
[PASS] arcxcore::tests::vt::points_claims::
test_vt_submit_zero_points_for_active_credit_period_allocates_no_claimable_points (l1_gas: ~0, l1_data_gas: ~9984,
↳ l2_gas: ~64983923)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_get_withdrawable_supply_excludes_recent_daily_inflows (l1_gas:
↳ ~0, l1_data_gas: ~5664, l2_gas: ~20349555)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_advance_fifo_head_with_max_id_stops_at_first_unburned_request
↳ (l1_gas: ~0, l1_data_gas: ~9216, l2_gas: ~57469201)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_get_withdrawable_supply_uses_max_lock_days_as_disable_sentinel
↳ (l1_gas: ~0, l1_data_gas: ~6816, l2_gas: ~29120175)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_locked_withdrawal_shares_cannot_be_transferred (l1_gas: ~0,
↳ l1_data_gas: ~6432, l2_gas: ~26339095)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_process_single_returns_none_when_already_burned (l1_gas: ~0,
↳ l1_data_gas: ~7968, l2_gas: ~32779365)
[PASS] arcxcore::tests::vt::withdraw_liquidity::
test_vt_process_single_withdrawal_can_process_out_of_order_but_fifo_head_stays_blocked (l1_gas: ~0, l1_data_gas: ~9792,
↳ l2_gas: ~69955351)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_process_withdrawal_processes_fifo_batch_until_target_id
↳ (l1_gas: ~0, l1_data_gas: ~9600, l2_gas: ~56970971)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_process_withdrawal_returns_none_if_request_exceeds_watermark
↳ (l1_gas: ~0, l1_data_gas: ~6432, l2_gas: ~24358845)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_process_withdrawal_rounds_maturity_time_up_to_bucket (l1_gas:
↳ ~0, l1_data_gas: ~7968, l2_gas: ~35856595)
[PASS]
↳ arcxcore::tests::vt::withdraw_liquidity::test_vt_process_withdrawal_uses_max_delay_as_immediate_maturity_sentinel
↳ (l1_gas: ~0, l1_data_gas: ~8256, l2_gas: ~38636805)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_processed_withdrawal_burns_locked_shares_and_mints_nft (l1_gas:
↳ ~0, l1_data_gas: ~7968, l2_gas: ~33526055)
[PASS] arcxcore::tests::vt::withdraw_liquidity::
test_vt_rebalance_liquidity_deposit_sends_usdc_to_bridge_and_updates_pending_deposit (l1_gas: ~0, l1_data_gas: ~6048,
↳ l2_gas: ~23990865)
[PASS] arcxcore::tests::vt::withdraw_liquidity::
test_vt_rebalance_liquidity_withdraw_sends_request_to_bridge_and_tracks_intransit (l1_gas: ~0, l1_data_gas: ~8448,
↳ l2_gas: ~41014325)
[PASS]
↳ arcxcore::tests::vt::withdraw_liquidity::test_vt_rebalance_liquidity_offsets_matured_withdrawal_and_makes_claimable
↳ (l1_gas: ~0, l1_data_gas: ~8352, l2_gas: ~42510345)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_rebalance_reverts_deposit_exceeds_pending (l1_gas: ~0,
↳ l1_data_gas: ~4800, l2_gas: ~9026969)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_rebalance_requires_bridge_when_bridge_withdraw_needed (l1_gas:
↳ ~0, l1_data_gas: ~7872, l2_gas: ~35236685)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_rebalance_reverts_when_withdrawal_exists_but_is_not_yet_matured
↳ (l1_gas: ~0, l1_data_gas: ~6528, l2_gas: ~26833425)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_rebalance_reverts_withdraw_exceeds_matured_pending (l1_gas: ~0,
↳ l1_data_gas: ~4800, l2_gas: ~9026969)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_redeem_withdrawal_burns_nft_and_transfers_usdc (l1_gas: ~0,
↳ l1_data_gas: ~8064, l2_gas: ~40298845)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_redeem_withdrawal_requires_nft_owner (l1_gas: ~0, l1_data_gas:
↳ ~8160, l2_gas: ~38298635)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_redeem_withdrawal_cannot_redeem_twice (l1_gas: ~0, l1_data_gas:
↳ ~8064, l2_gas: ~41367205)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_redeem_withdrawal_respects_cumulative_usdc_fifo (l1_gas: ~0,
↳ l1_data_gas: ~8928, l2_gas: ~50591175)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_redeem_withdrawal_reverts_before_usdc_is_claimable (l1_gas: ~0,
↳ l1_data_gas: ~7968, l2_gas: ~34990085)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_redeem_withdrawal_transfers_usdc_to_current_nft_owner (l1_gas:
↳ ~0, l1_data_gas: ~8064, l2_gas: ~42620065)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_request_withdrawal_reverts_insufficient_balance (l1_gas: ~0,
↳ l1_data_gas: ~5664, l2_gas: ~32898135)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_request_withdrawal_locks_shares_and_snapshots_fifo (l1_gas: ~0,
↳ l1_data_gas: ~6720, l2_gas: ~26732435)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_request_withdrawal_reverts_zero_shares (l1_gas: ~0,
↳ l1_data_gas: ~5664, l2_gas: ~32701425)
[PASS] arcxcore::tests::core_admin::test_unpause_only_owner (l1_gas: ~0, l1_data_gas: ~1344, l2_gas: ~2409790)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_unlocked_remainder_stays_transferable_after_withdrawal_request
↳ (l1_gas: ~0, l1_data_gas: ~6720, l2_gas: ~27799495)
```

```
[PASS] arcxcore::tests::core_admin::test_unpause_st_ept_only_owner (l1_gas: ~0, l1_data_gas: ~10272, l2_gas: ~19968992)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_operator_requires_admin (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~11176481)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_role_update_rejects_zero_operator (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~11171871)
[PASS] arcxcore::tests::vt::withdraw_liquidity::test_vt_owner_can_withdraw_accrued_nav_performance_fees_through_redeem
↳ (l1_gas: ~0, l1_data_gas: ~175392, l2_gas: ~1127123217)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_rebalance_allows_operator_or_admin_only (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~12637101)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_role_update_requires_admin (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~11171971)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_performance_fees_rejects_nav_fee_above_100pct (l1_gas: ~0,
↳ l1_data_gas: ~6336, l2_gas: ~12404441)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_performance_fees_rejects_points_fee_above_100pct (l1_gas:
↳ ~0, l1_data_gas: ~6336, l2_gas: ~12404441)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_performance_fees_rejects_when_paused (l1_gas: ~0,
↳ l1_data_gas: ~6432, l2_gas: ~12587691)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_bridge_config_rejects_zero (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~11171871)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_bridge_config_requires_admin (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~11176481)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_performance_fees_requires_admin (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~12401101)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_performance_fees_updates_config (l1_gas: ~0, l1_data_gas:
↳ ~6528, l2_gas: ~11839381)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_deposit_lock_days_requires_admin (l1_gas: ~0, l1_data_gas:
↳ ~6336, l2_gas: ~11167271)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_treasury_rejects_zero (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~11171871)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_deposit_lock_days_updates_value (l1_gas: ~0, l1_data_gas:
↳ ~6432, l2_gas: ~11552161)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_treasury_requires_admin (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~11176481)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_operator_rejects_zero (l1_gas: ~0, l1_data_gas: ~6336,
↳ l2_gas: ~11171871)
[PASS] arcxcore::tests::vt::constructor_admin::test_vt_set_withdraw_batch_delay_rejects_pending_withdrawals (l1_gas:
↳ ~0, l1_data_gas: ~9792, l2_gas: ~41998727)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_trigger_withdrawal_request_rejects_outsider (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15255137)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_redeem_vtoken_rejects_unregistered_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~10272, l2_gas: ~16314407)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_trigger_withdrawal_request_rejects_unregistered_vtoken
↳ (l1_gas: ~0, l1_data_gas: ~10272, l2_gas: ~16217817)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_trigger_withdrawal_request_rejects_registry_pause
↳ (l1_gas: ~0, l1_data_gas: ~9888, l2_gas: ~16132397)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_redeem_vtoken_rejects_zero_vtoken (l1_gas: ~0,
↳ l1_data_gas: ~9792, l2_gas: ~15306627)
[PASS] arcxcore::tests::core_admin::test_upgrade_happy_path_owner_uses_same_class (l1_gas: ~0, l1_data_gas: ~1248,
↳ l2_gas: ~2105430)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_trigger_withdrawal_request_rejects_local_pause (l1_gas:
↳ ~0, l1_data_gas: ~9888, l2_gas: ~16014567)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_trigger_withdrawal_request_rejects_zero_vtoken (l1_gas:
↳ ~0, l1_data_gas: ~9792, l2_gas: ~15210037)
[PASS] arcxcore::tests::core_admin::test_update_registry_rejects_non_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~1689440)
[PASS] arcxcore::tests::core_admin::test_update_registry_happy_path_and_getter (l1_gas: ~0, l1_data_gas: ~7104, l2_gas:
↳ ~12366791)
[PASS] arcxcore::tests::core_admin::test_update_max_order_apr_bps_rejects_zero (l1_gas: ~0, l1_data_gas: ~1248, l2_gas:
↳ ~2024370)
```



```
[PASS] arcxcore::tests::core_admin::test_update_st_token_class_only_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1578980)
[PASS] arcxcore::tests::core_admin::test_update_registry_rejects_zero (l1_gas: ~0, l1_data_gas: ~7104, l2_gas: ~11863111)
[PASS] arcxcore::tests::core_admin::test_update_fee_treasury_happy_path (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~2393090)
[PASS] arcxcore::tests::core_admin::test_upgrade_ept_token_only_owner (l1_gas: ~0, l1_data_gas: ~10176, l2_gas: ~18993492)
[PASS] arcxcore::tests::core_admin::test_update_fee_treasury_only_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1780480)
[PASS] arcxcore::tests::core_admin::test_update_fee_treasury_rejects_zero (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~2028000)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_withdrawal_request_and_redeem_use_direct_vtoken_address (l1_gas: ~0, l1_data_gas: ~14880, l2_gas: ~68434873)
[PASS] arcxcore::tests::core_admin::test_upgrade_only_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1596870)
[PASS] arcxcore::tests::core_admin::test_update_max_order_apr_bps_only_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1776850)
[PASS] arcxcore::tests::core_admin::test_update_max_order_apr_bps_happy_path (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~2389560)
[PASS] arcxcore::tests::core_admin::test_upgrade_st_token_only_owner (l1_gas: ~0, l1_data_gas: ~10176, l2_gas: ~18993492)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_respects_apr_cap_on_limit (l1_gas: ~0, l1_data_gas: ~11136, l2_gas: ~33037422)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_reverts_on_partial_fill (l1_gas: ~0, l1_data_gas: ~11136, l2_gas: ~33612052)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_happy_path_st_buy (l1_gas: ~0, l1_data_gas: ~15744, l2_gas: ~95728091)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_happy_path_st_sell (l1_gas: ~0, l1_data_gas: ~15744, l2_gas: ~95741391)
[PASS] arcxcore::tests::core_admin::test_update_ept_token_class_only_owner (l1_gas: ~0, l1_data_gas: ~1248, l2_gas: ~1578980)
[PASS] arcxcore::tests::vault::points_withdrawals::test_vault_redeem_vtoken_rejects_registry_pause (l1_gas: ~0, l1_data_gas: ~9888, l2_gas: ~16228987)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_happy_path_ept_buy (l1_gas: ~0, l1_data_gas: ~16704, l2_gas: ~119267480)
[PASS] arcxcore::tests::core_invariants::test_strategy_collateralization_holds_across_mint_burn_cycle (l1_gas: ~0, l1_data_gas: ~12384, l2_gas: ~215195682)
[PASS] arcxcore::tests::core_invariants::test_vtoken_strategies_vec_retains_all_children (l1_gas: ~0, l1_data_gas: ~18720, l2_gas: ~147602641)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_respects_apr_cap_on_quoted (l1_gas: ~0, l1_data_gas: ~11136, l2_gas: ~33080642)
[PASS] arcxcore::tests::core_market::test_create_market_and_match_happy_path_ept_sell (l1_gas: ~0, l1_data_gas: ~16800, l2_gas: ~119257050)
Tests: 526 passed, 0 failed, 0 ignored, 0 filtered out
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.